



AGILE DEVELOPMENT PROCESS AND DEVOPS (ISWE406L) COURSE PROJECT

CASE STUDY:

HOSPITAL MANAGEMENT SYSTEM

IFFAH SHAFEEQUE ETTUVEETIL

23MIS0072

INTRODUCTION

This case study presents the development of a Hospital Management System using an Agile and DevOps-based approach. The system was implemented in three incremental versions, where each version introduced a new feature and extended the functionality of the previous one. The project emphasizes not only application development but also the integration of modern DevOps practices such as version control, containerization, continuous integration, and deployment. Each version was developed in VS Code, managed using Git and GitHub with separate branches, and processed through Docker and Jenkins pipelines. Deployment was further supported using Kubernetes configurations to demonstrate an end-to-end development workflow. This structured approach highlights the practical application of Agile and DevOps principles in building a scalable system.

IMPLEMENTATION DETAILS AND REQUIREMENT MAPPING

This section describes how each of the specified project requirements was implemented throughout the case study, covering both development and DevOps aspects.

1. Developed in VS Code

The application was developed entirely using Visual Studio Code. A structured project layout was created including backend (Flask), frontend templates (HTML), styling (CSS), and configuration files. Each version (v1, v2, v3) was incrementally developed by modifying and extending the existing codebase within the same environment.

Before integrating with version control, all versions were tested locally using:

python app.py

The application was accessed via:

<http://127.0.0.1:5000>

This ensured that each version was functionally correct before proceeding to further stages.

2. Pushed to GitHub (Separate Commits and Branches)

Version control was maintained using Git, following an incremental branching strategy:

- main branch → baseline project
- v1-feature-add-patient → Version 1
- v2-feature-view-patient → Version 2 (created from v1)
- v3-feature-delete-patient → Version 3 (created from v2)

Each version was committed separately with meaningful commit messages and pushed to GitHub. This ensured clear separation of features and proper traceability of development.

3. Built Using Jenkins Pipeline

A Jenkins pipeline was configured to automate the build process for each version. The pipeline included the following stages:

- Source code checkout from GitHub
- Docker image build for the respective version
- Push of the built image to DockerHub

This ensured that the build process was automated and reproducible for all versions.

4. Containerized Using Docker (Tagged v1, v2, v3)

Each version of the application was containerized using Docker. Separate images were created and tagged according to the version:

- hospital-system:v1
- hospital-system:v2
- hospital-system:v3

Before integrating with Jenkins, Docker images were manually built and tested using:

docker build -t image-name .

docker run -p <port>:5000 image-name

This provided an additional layer of verification to ensure the application runs correctly inside a container.

5. Pushed to DockerHub

The Docker images for each version were pushed to DockerHub through the Jenkins pipeline. This ensured centralized storage of versioned images and allowed them to be used for deployment.

Each version (v1, v2, v3) is available as a separate tagged image in the DockerHub repository.

6. Deployed in Kubernetes as Separate Deployments and Services

Kubernetes deployment was performed locally using Docker Desktop Kubernetes. Each version was deployed using its own:

- Deployment YAML file
- Service YAML file

Separate configurations were maintained for each version to ensure isolation and clarity.

Each version was assigned a unique NodePort:

- Version 1 → Port 30081
- Version 2 → Port 30082
- Version 3 → Port 30083

This allowed all versions to run independently without conflicts.

7. Accessible via Localhost (Different Ports)

Each version of the application was made accessible through different ports to allow simultaneous testing:

Version	Access Method	Port
v1	Docker	localhost:5001
v2	Docker	localhost:5002
v3	Docker	localhost:5003
v1	Kubernetes	localhost:30081
v2	Kubernetes	localhost:30082
v3	Kubernetes	localhost:30083

This ensured that each version could be tested independently and verified without affecting other versions.

IMPLEMENTATION

VERSION 1 – ADD PATIENT

Version 1 implements the core functionality of the system, which allows users to register patient details through a web interface. The entered data is stored in a structured format for further use in later versions.

Phase 1 — Project Setup in VS Code

The initial project structure was created in VS Code to organize backend, frontend, and configuration files. This ensured a clean and modular setup, making it easier to extend the system in future versions.

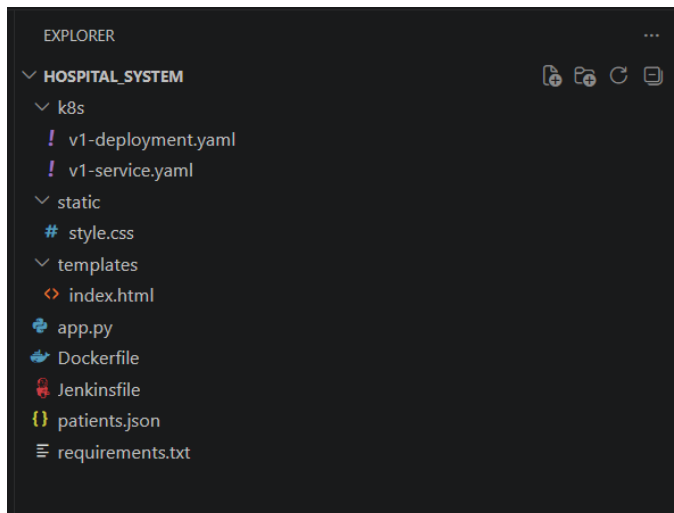


Figure 1: Version 1 project structure created in VS Code

Phase 2 — Application Development

The backend, frontend, and data storage components were developed in this phase. A Flask-based backend was created to handle form submissions, while HTML and CSS were used to design the user interface. A JSON file was initialized to store patient data persistently.

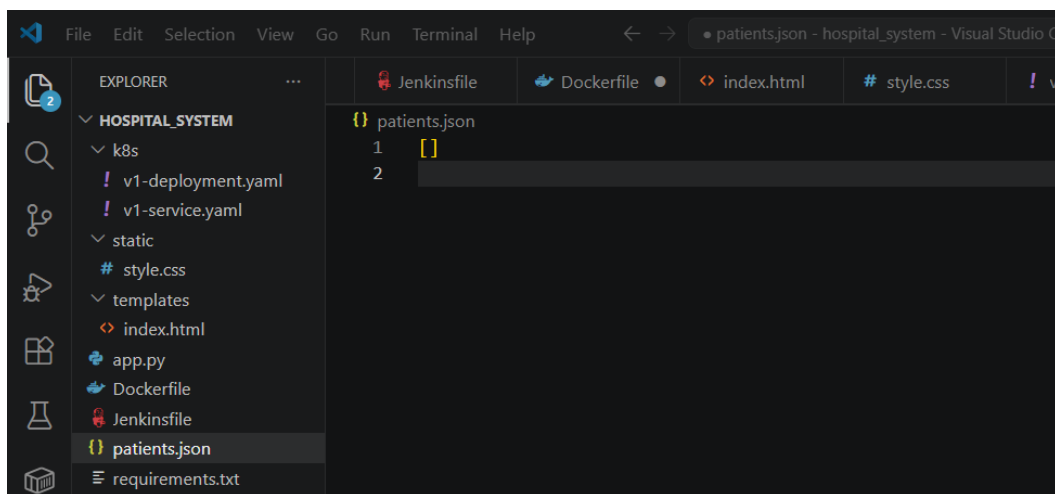


Figure 2: patients.json initialized

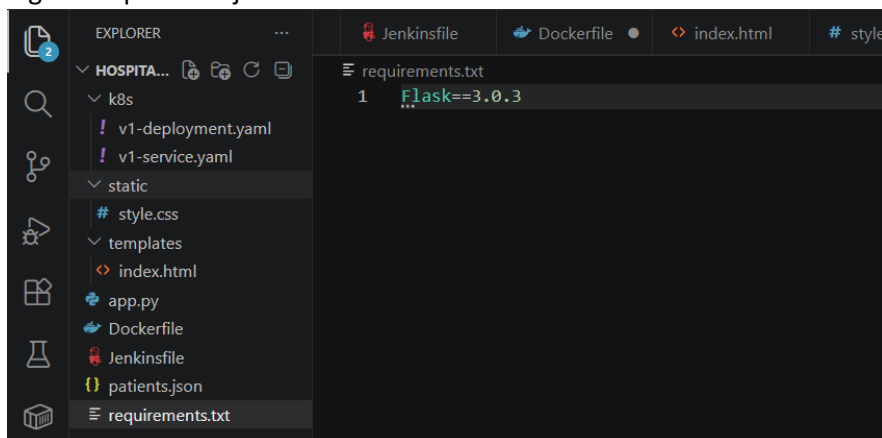
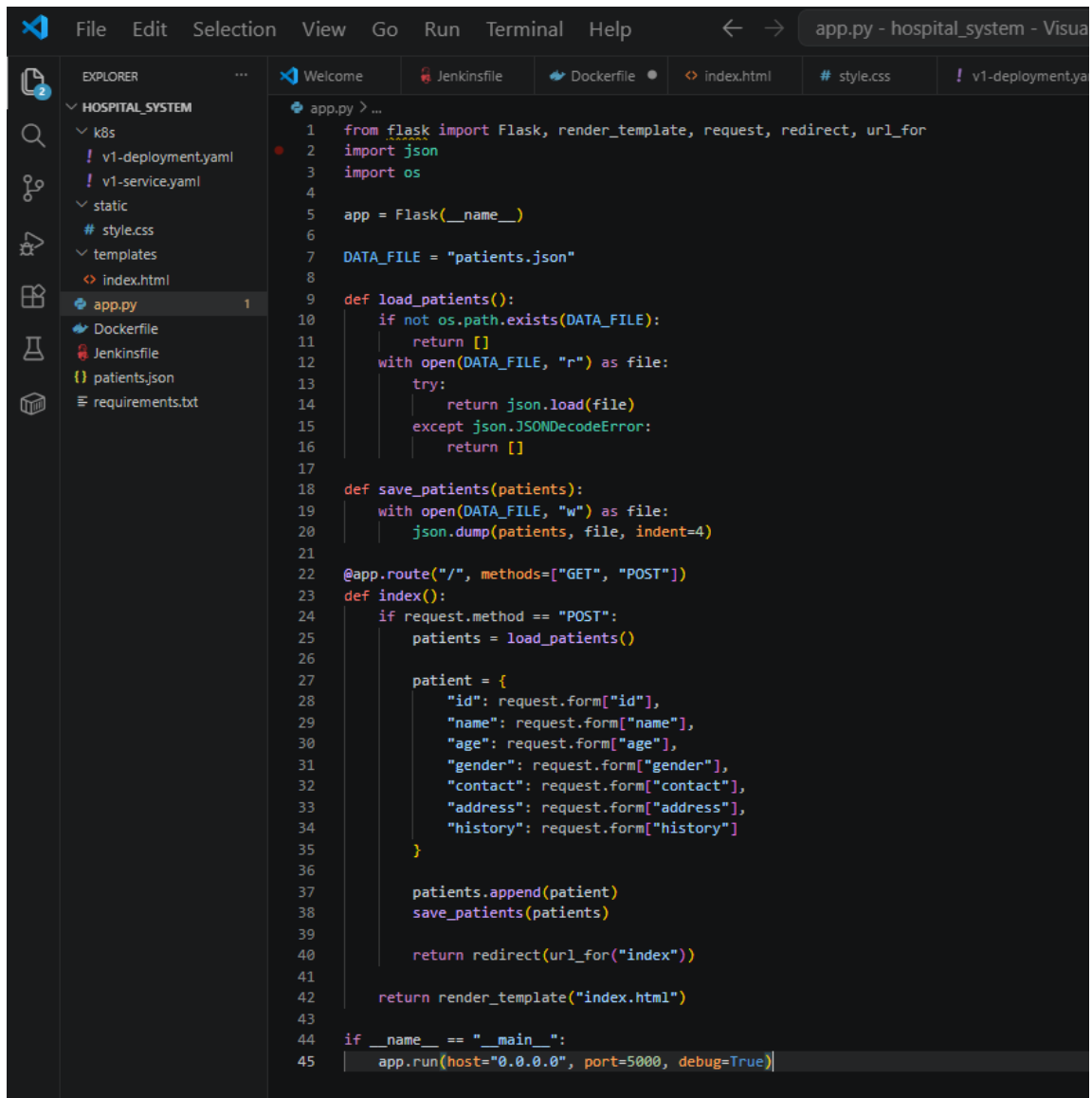


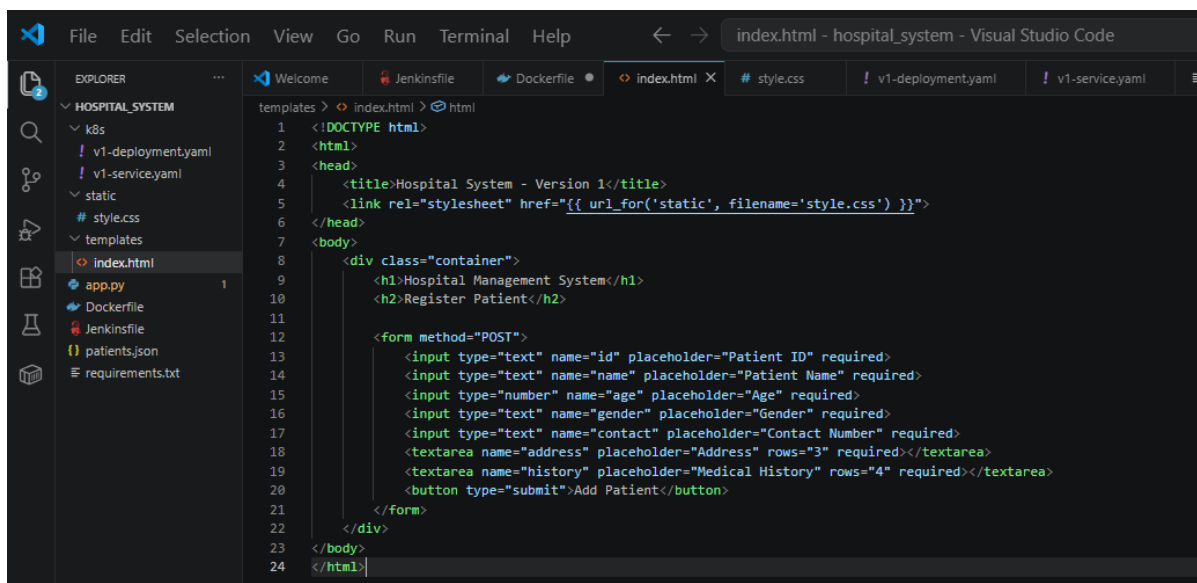
Figure 3: requirements.txt initialized



The screenshot shows the Visual Studio Code editor with the file explorer on the left displaying the project structure for 'HOSPITAL_SYSTEM'. The main editor window shows the 'app.py' file, which is a Flask application. The code includes imports for Flask, json, and os. It defines a Flask app, a data file path, and two functions: 'load_patients()' and 'save_patients()'. A route is defined for the root path, handling GET and POST requests. The POST handler loads patients, creates a new patient object from form data, appends it to the list, saves it, and redirects to the index. The app runs on host '0.0.0.0' and port 5000.

```
1 from flask import Flask, render_template, request, redirect, url_for
2 import json
3 import os
4
5 app = Flask(__name__)
6
7 DATA_FILE = "patients.json"
8
9 def load_patients():
10     if not os.path.exists(DATA_FILE):
11         return []
12     with open(DATA_FILE, "r") as file:
13         try:
14             return json.load(file)
15         except json.JSONDecodeError:
16             return []
17
18 def save_patients(patients):
19     with open(DATA_FILE, "w") as file:
20         json.dump(patients, file, indent=4)
21
22 @app.route("/", methods=["GET", "POST"])
23 def index():
24     if request.method == "POST":
25         patients = load_patients()
26
27         patient = {
28             "id": request.form["id"],
29             "name": request.form["name"],
30             "age": request.form["age"],
31             "gender": request.form["gender"],
32             "contact": request.form["contact"],
33             "address": request.form["address"],
34             "history": request.form["history"]
35         }
36
37         patients.append(patient)
38         save_patients(patients)
39
40         return redirect(url_for("index"))
41
42     return render_template("index.html")
43
44 if __name__ == "__main__":
45     app.run(host="0.0.0.0", port=5000, debug=True)
```

Figure 4: Flask backend for Version 1 patient registration



The screenshot shows the Visual Studio Code editor with the file explorer on the left displaying the project structure for 'HOSPITAL_SYSTEM'. The main editor window shows the 'index.html' file, which is an HTML form for patient registration. The form includes a title 'Hospital System - Version 1', a link to the stylesheet, and a form with input fields for id, name, age, gender, contact, address, and history, and a submit button labeled 'Add Patient'.

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4     <title>Hospital System - Version 1</title>
5     <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
6 </head>
7 <body>
8     <div class="container">
9         <h1>Hospital Management System</h1>
10        <h2>Register Patient</h2>
11
12        <form method="POST">
13            <input type="text" name="id" placeholder="Patient ID" required>
14            <input type="text" name="name" placeholder="Patient Name" required>
15            <input type="number" name="age" placeholder="Age" required>
16            <input type="text" name="gender" placeholder="Gender" required>
17            <input type="text" name="contact" placeholder="Contact Number" required>
18            <textarea name="address" placeholder="Address" rows="3" required></textarea>
19            <textarea name="history" placeholder="Medical History" rows="4" required></textarea>
20            <button type="submit">Add Patient</button>
21        </form>
22    </div>
23 </body>
24 </html>
```

Figure 5: HTML form for Version 1 patient registration

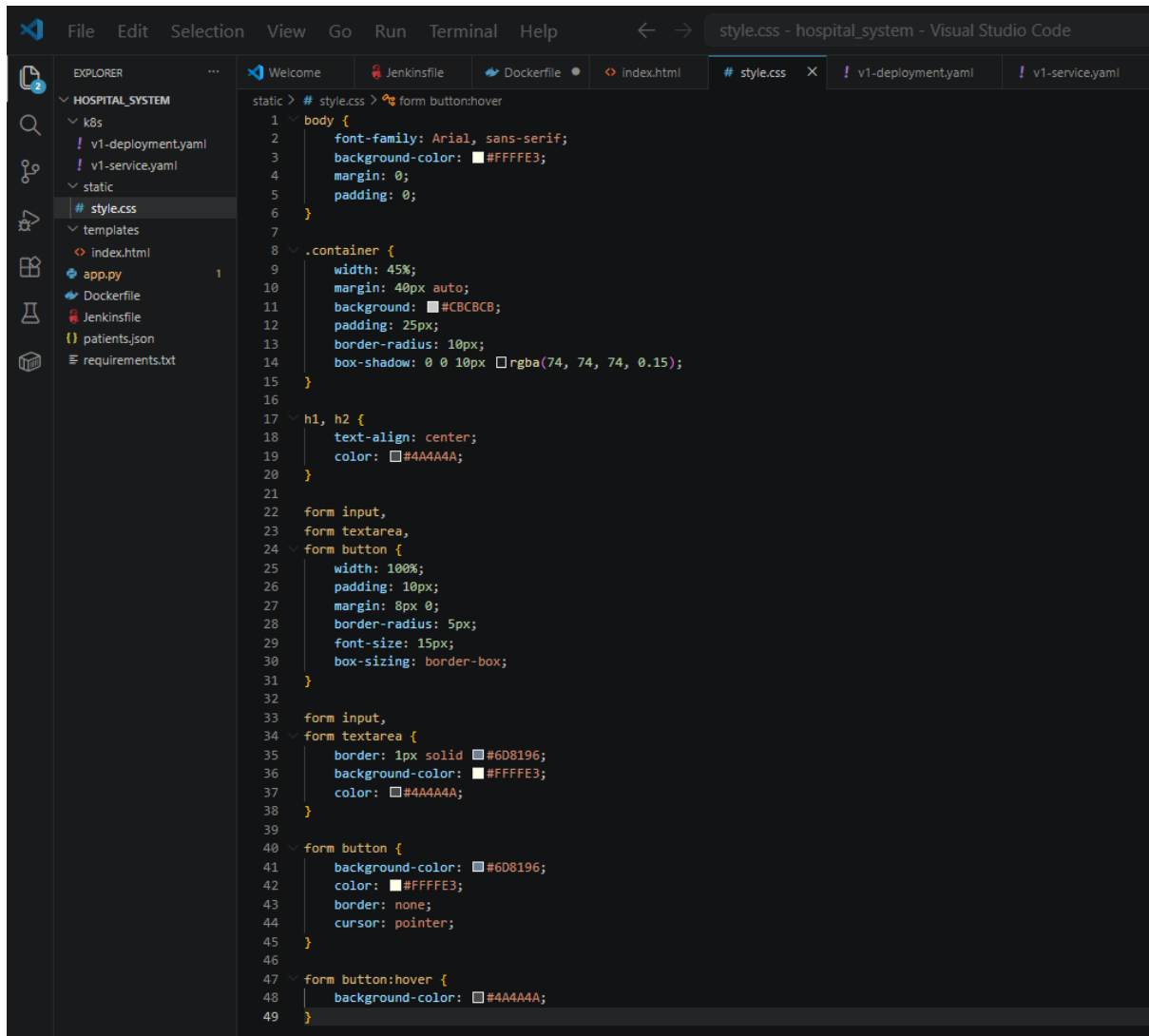


Figure 6: CSS styling for Version 1 interface

Phase 3 — Local Testing

The application was executed locally on <http://127.0.0.1:5000> using Flask to verify that patient data could be entered and stored correctly. This step ensured that the system was functioning properly before proceeding to version control and deployment stages.

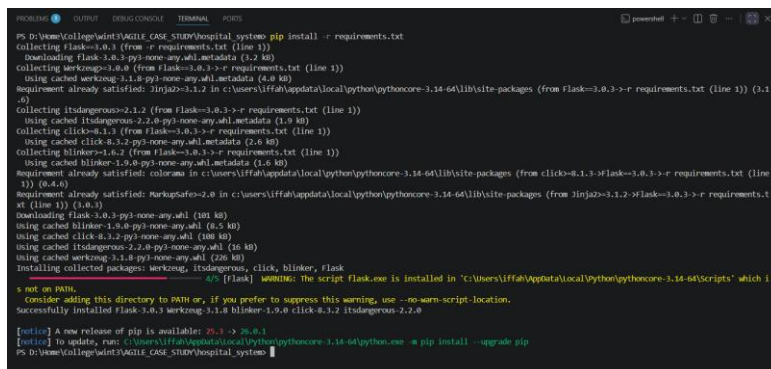


Figure 7: Dependencies installed for Version 1

```
PS D:\Home\College\wint3\AGILE_CASE_STUDY\hospital_system> python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.1.41:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 112-728-999
█
```

Figure 8: Flask application running locally

The screenshot shows a web browser window titled "Hospital System - Version 1". The address bar shows "192.168.1.41:5000". The page displays a registration form for a "Hospital Management System". The form is titled "Register Patient" and contains the following fields: "Patient ID", "Patient Name", "Age", "Gender", "Contact Number", "Address", and "Medical History". Below these fields is a blue "Add Patient" button. The form is set against a light yellow background.

Figure 9: Version 1 registration form opened in browser

The screenshot shows the same registration form as in Figure 9, but with sample patient details entered. The "Patient ID" field contains "P1001", "Patient Name" contains "Iffah Shafeeqe", "Age" contains "22", "Gender" contains "Female", "Contact Number" contains "9993728292", "Address" contains "Kozhikode, Kerala", and "Medical History" contains "Diabetes, seasonal allergies". The "Add Patient" button is still visible at the bottom.

Figure 10: Sample patient details entered


```

1  [
2
3      {
4          "id": "P1001",
5          "name": "Iffah Shafeeqe",
6          "age": "22",
7          "gender": "Female",
8          "contact": "9993728292",
9          "address": "Kozhikode, Kerala",
10         "history": "Diabetes, seasonal allergies"
11     }
12 ]

```

Figure 11: Patient data stored successfully in patients.json

Phase 4 — Git Initialization

The project was initialized as a Git repository to enable version control. An initial commit was created on the main branch to establish a stable baseline for further development.

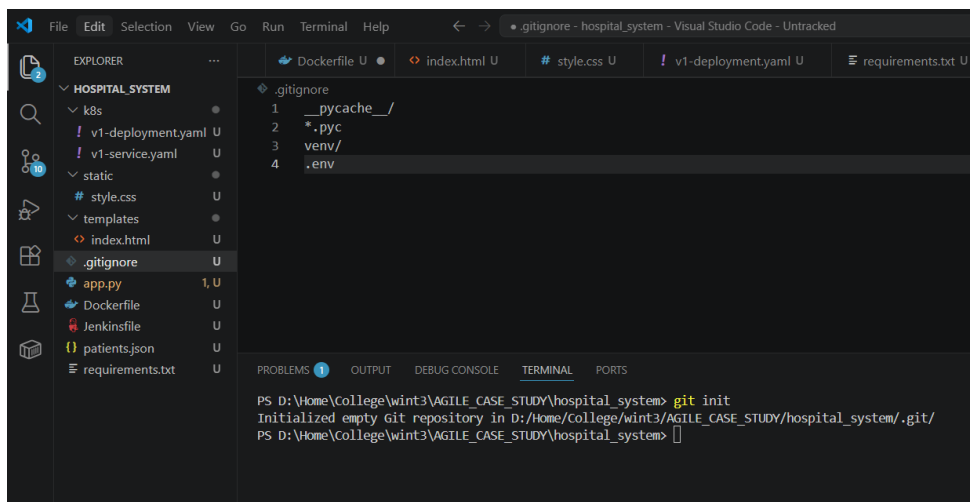


Figure 12: Git repository initialized

```

PS D:\Home\College\wint3\AGILE_CASE_STUDY\hospital_system> git init
Initialized empty Git repository in D:\Home\College\wint3\AGILE_CASE_STUDY\hospital_system\.git\
PS D:\Home\College\wint3\AGILE_CASE_STUDY\hospital_system> git add .
PS D:\Home\College\wint3\AGILE_CASE_STUDY\hospital_system> git commit -m "chore: initialize hospital system project structure and v1 base app"
[master (root-commit) aed60e4] chore: initialize hospital system project structure and v1 base app
10 files changed, 131 insertions(+)
create mode 100644 .gitignore
create mode 100644 Dockerfile
create mode 100644 Jenkinsfile
create mode 100644 app.py
create mode 100644 k8s/v1-deployment.yaml
create mode 100644 k8s/v1-service.yaml
create mode 100644 patients.json
create mode 100644 requirements.txt
create mode 100644 static/style.css
create mode 100644 templates/index.html
PS D:\Home\College\wint3\AGILE_CASE_STUDY\hospital_system> git branch -M main
PS D:\Home\College\wint3\AGILE_CASE_STUDY\hospital_system>

```

Figure 13: Initial baseline commit created on main branch

Phase 5 — Version 1 Branch Creation

A separate branch was created for Version 1 to isolate the development of the Add Patient feature. This ensures proper version management and follows Agile development practices.

```
PS D:\Home\College\wint3\AGILE_CASE_STUDY\hospital_system> git checkout -b v1-feature-add-patient
Switched to a new branch 'v1-feature-add-patient'
PS D:\Home\College\wint3\AGILE_CASE_STUDY\hospital_system> █
```

Figure 14: Separate Version 1 branch created

```
PS D:\Home\College\wint3\AGILE_CASE_STUDY\hospital_system> git branch
main
* v1-feature-add-patient
PS D:\Home\College\wint3\AGILE_CASE_STUDY\hospital_system> █
```

Figure 15: Branch list showing main and v1-feature-add-patient

Phase 6 — DevOps File Integration

Deployment-related files such as Dockerfile, Kubernetes manifests, and Jenkins pipeline configuration were added. These files enable containerization, automation, and deployment of the application.

```
Dockerfile > ...
1 FROM python:3.11-slim
2
3 WORKDIR /app
4
5 COPY requirements.txt .
6 RUN pip install --no-cache-dir --no-compile --progress-bar off -r requirements.txt
7
8 COPY . .
9
10 EXPOSE 5000
11
12 CMD ["python", "app.py"]
```

Figure 16: Dockerfile added for Version 1

```
k8s > ! v1-deployment.yaml
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: hospital-v1
5 spec:
6   replicas: 1
7   selector:
8     matchLabels:
9       app: hospital-v1
10  template:
11    metadata:
12      labels:
13        app: hospital-v1
14    spec:
15      containers:
16        - name: hospital-v1
17          image: iffah11/hospital-system:v1
18          imagePullPolicy: Always
19          ports:
20            - containerPort: 5000
```

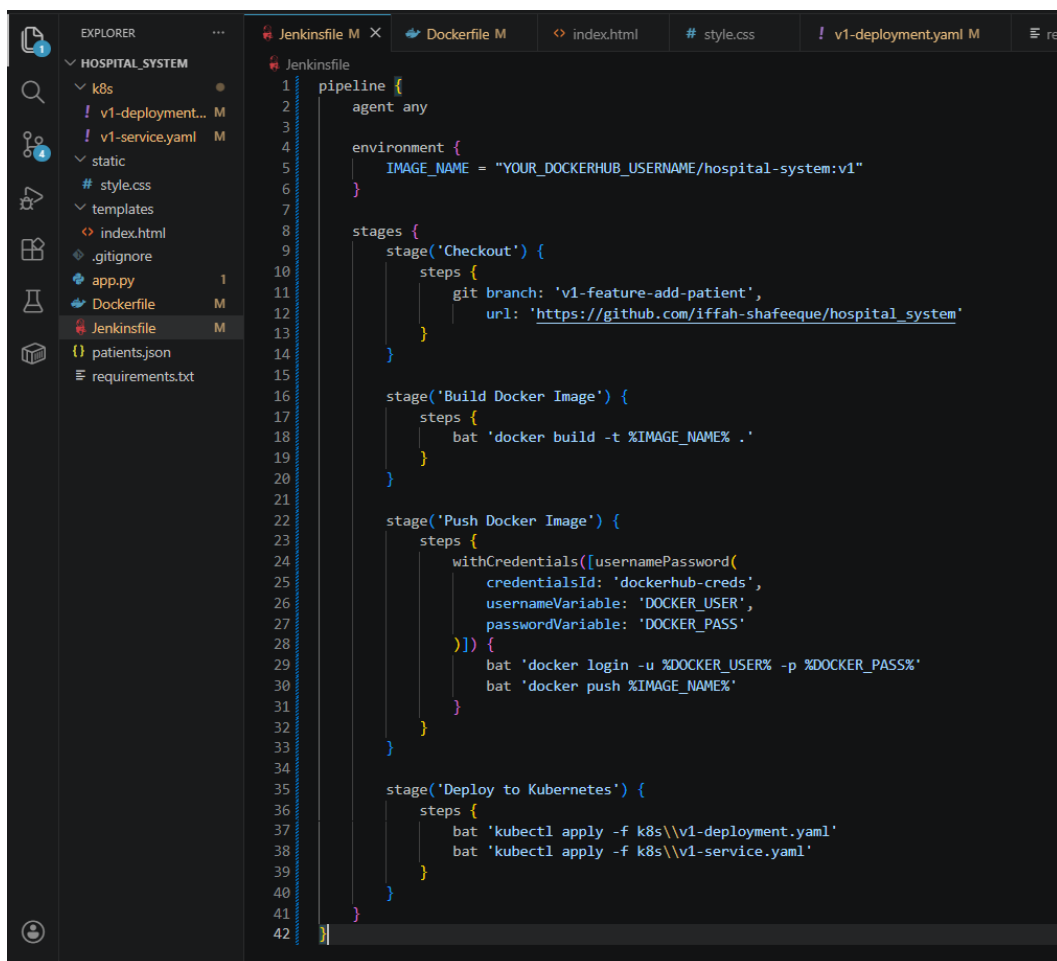
Figure 17: Kubernetes deployment.yaml for Version 1

```

k8s > ! v1-service.yaml
1  apiVersion: v1
2  kind: Service
3  metadata:
4    name: hospital-v1-service
5  spec:
6    type: NodePort
7    selector:
8      app: hospital-v1
9    ports:
10     - protocol: TCP
11       port: 5000
12       targetPort: 5000
13       nodePort: 30081

```

Figure 18: Kubernetes service.yaml for Version 1



```

Jenkinsfile
1  pipeline {
2    agent any
3
4    environment {
5      IMAGE_NAME = "YOUR_DOCKERHUB_USERNAME/hospital-system:v1"
6    }
7
8    stages {
9      stage('Checkout') {
10       steps {
11         git branch: 'v1-feature-add-patient',
12           url: 'https://github.com/iffah-shafeeqe/hospital_system'
13       }
14     }
15
16     stage('Build Docker Image') {
17       steps {
18         bat 'docker build -t %IMAGE_NAME% .'
19       }
20     }
21
22     stage('Push Docker Image') {
23       steps {
24         withCredentials([usernamePassword(
25           credentialsId: 'dockerhub-creds',
26           usernameVariable: 'DOCKER_USER',
27           passwordVariable: 'DOCKER_PASS'
28         )]) {
29           bat 'docker login -u %DOCKER_USER% -p %DOCKER_PASS%'
30           bat 'docker push %IMAGE_NAME%'
31         }
32       }
33     }
34
35     stage('Deploy to Kubernetes') {
36       steps {
37         bat 'kubectl apply -f k8s\\v1-deployment.yaml'
38         bat 'kubectl apply -f k8s\\v1-service.yaml'
39       }
40     }
41   }
42 }

```

Figure 19: Jenkins pipeline definition added

```

PS D:\Home\College\wint3\AGILE_CASE_STUDY\hospital_system> git add Dockerfile Jenkinsfile k8s/ .gitignore
PS D:\Home\College\wint3\AGILE_CASE_STUDY\hospital_system> git commit -m "ci(v1): add Docker, Jenkins pipeline, and Kubernetes manifests for Version 1"
[v1-feature-add-patient a73dcf1] ci(v1): add Docker, Jenkins pipeline, and Kubernetes manifests for Version 1
4 files changed, 86 insertions(+), 1 deletion(-)
PS D:\Home\College\wint3\AGILE_CASE_STUDY\hospital_system>

```

Figure 20: DevOps files committed

Phase 7 — GitHub Integration

The project was pushed to GitHub to enable remote version control and collaboration. The main branch and Version 1 branch were uploaded along with their respective commits.

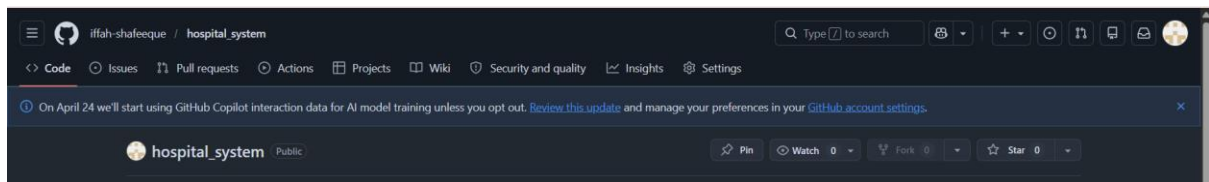


Figure 21: GitHub repository created

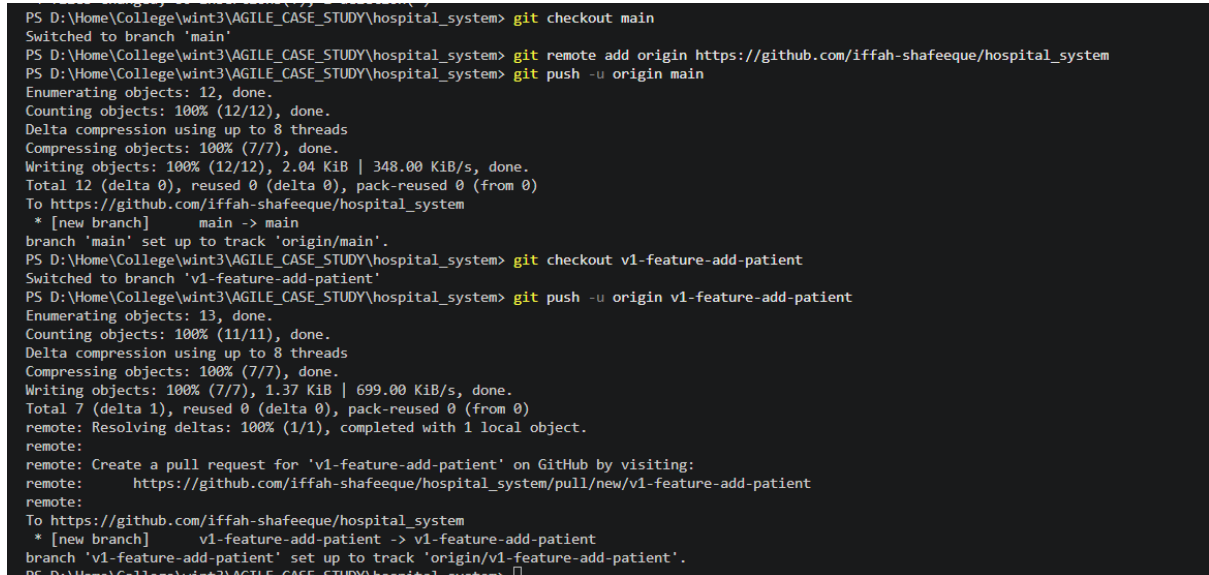


Figure 22: Branches pushed to GitHub

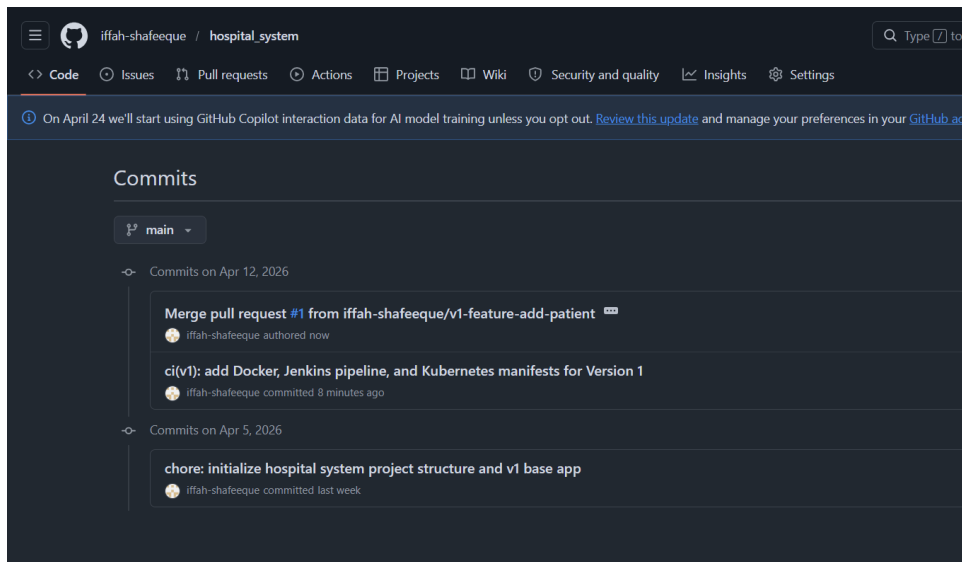


Figure 23: Commit history for Version 1

Phase 8 — Docker Build and Test

The application was containerized using Docker and built into an image tagged as v1. The container was then executed locally to verify that the application runs correctly in a containerized environment.

```
PS D:\Home\College\wint3\AGILE_CASE_STUDY\hospital_system> docker build -t iffah11/hospital-system:v1 .
[+] Building 48.8s (11/11) FINISHED
    => [internal] load build definition from Dockerfile
    => => transferring dockerfile: 210B
    => [internal] load metadata for docker.io/library/python:3.11-slim
    => [auth] library/python:pull token for registry-1.docker.io
    => [internal] load .dockerignore
    => => transferring context: 2B
    => [internal] load build context
    => => transferring context: 45.72kB
    => [1/5] FROM docker.io/library/python:3.11-slim@sha256:233de06753d30d120b1a3ce359d8d3be8bda78524cd8f520c99883bfe33964cf
    => resolve docker.io/library/python:3.11-slim@sha256:233de06753d30d120b1a3ce359d8d3be8bda78524cd8f520c99883bfe33964cf
    => sha256:d98f36ba21593a4014243f41ec9c5370abd34f4f65a5d074e41bce23f5f6a165 14.37MB / 14.37MB
    => sha256:0816ee351b60d03c7a7a3f50c717891eed927f4e6a1d59f6ce05268065606 251B / 251B
    => sha256:37ffa6577b04e98598e2c10432eb8c66cfc1af6c7698b21c6163f8334606aa14 1.29MB / 1.29MB
    => sha256:5435b2dcdf547f4a0d01d445dbec27a776fab9a605336f5067d6e9ecb5976 29.70MB / 29.70MB
    => extracting sha256:5435b2dcdf547f4a0d01d445dbec27a776fab9a605336f5067d6e9ecb5976
    => extracting sha256:37ffa6577b04e98598e2c10432eb8c66cfc1af6c7698b21c6163f8334606aa14
    => extracting sha256:d98f36ba21593a4014243f41ec9c5370abd34f4f65a5d074e41bce23f5f6a165
    => extracting sha256:0816ee351b60d03c7a7a3f50c717891eed927f4e6a1d59f6ce05268065606
    => [2/5] WORKDIR /app
    => [3/5] COPY requirements.txt .
    => [4/5] RUN pip install --no-cache-dir -r requirements.txt
    => [5/5] COPY . .
    => exporting image
    => exporting layers
    => exporting manifest sha256:6763883655b6a8f7b82e0255b9654757ef48a2f5c32e172c76ee25c809a0e699
    => exporting config sha256:4bc3587a54d08925b84295cd4f517ffae7d83c231469433043739122abc6434
    => exporting attestation manifest sha256:76550a8925e16a2e9d080c8a993bacb888afa557aa7d85b491ea384ac0931a0
    => exporting manifest list sha256:cc1581f8b4637087d4f39b3a091e04eb5f831f934b466e981886d863fe3fcfe
    => naming to docker.io/iffah11/hospital-system:v1
    => unpacking to docker.io/iffah11/hospital-system:v1

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/yth5vry2woych8wzxkz51t2537
```

Figure 24: Docker image built successfully

```
PS D:\Home\College\wint3\AGILE_CASE_STUDY\hospital_system> docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
iffah11/hospital-system	v1	cc1581f8b463	20 seconds ago	211MB
docker/desktop-kubernetes	kubernetes-v1.31.4-cni-v1.6.0-critools-v1.31.1-cri-dockerd-v0.3.15-1-debian	1405239673ef	16 months ago	577MB
registry.k8s.io/kube-apiserver	v1.31.4	ace6a943b058	16 months ago	125MB
registry.k8s.io/kube-controller-manager	v1.31.4	4bd1d4a449e7	16 months ago	118MB
registry.k8s.io/kube-scheduler	v1.31.4	1a3081cb7d21	16 months ago	90.8MB
registry.k8s.io/kube-proxy	v1.31.4	1739b3febca3	16 months ago	126MB
registry.k8s.io/etcd	3.5.16-0	c6a9d11cc5c0	19 months ago	211MB
registry.k8s.io/coredns/coredns	v1.11.3	9caabfb6238b	20 months ago	85.1MB
registry.k8s.io/etcd	3.5.15-0	a6dc63e6e8cf	20 months ago	208MB
registry.k8s.io/pause	3.10	ee6521f29bb2	22 months ago	1.06MB
docker/desktop-vpnkit-controller	dc331cb22850be0cdd97c84a9cfecaf44a1afb6e	7ecf567ea070	2 years ago	47MB
docker/desktop-storage-provisioner	v2.0	115d77efe6e2	4 years ago	59.2MB

Figure 25: Docker image tagged as v1

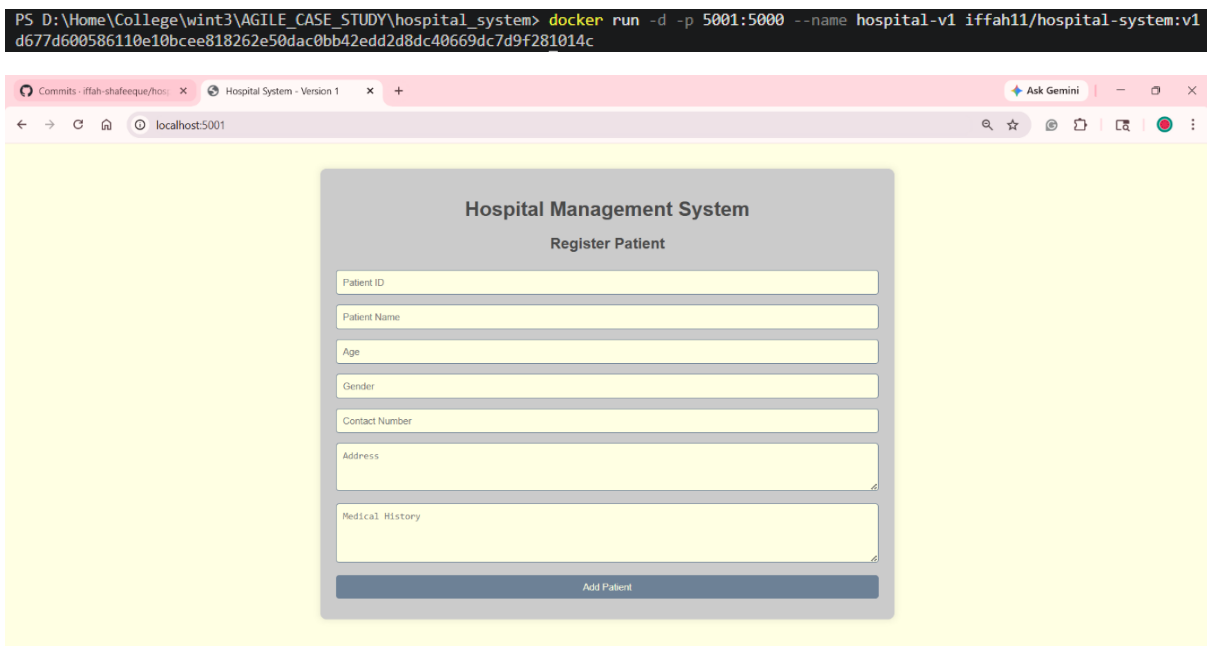


Figure 26: Application running on localhost:5001

Phase 9 — Jenkins Pipeline

A Jenkins pipeline was configured to automate the build process. The pipeline successfully fetched the code, built the Docker image, and pushed it to DockerHub. Due to system limitations and Docker Desktop instability, the Kubernetes deployment was not automated using Jenkins and shown here manually done after the successful creation of docker image in the local system.

The screenshot shows the Jenkins 'Configure' page for a pipeline named 'IffahApp-V1'. The 'Pipeline' tab is selected, showing the 'Pipeline script' section. The script is a declarative pipeline with the following stages:

```
1 pipeline {
2   agent any
3
4   environment {
5     IMAGE_NAME = "iffah11/hospital-system:v1"
6   }
7
8   stages {
9     stage('Checkout') {
10      steps {
11        git branch: 'v1-feature-add-patient',
12          url: 'https://github.com/iffah-shafeeqe/hospital_system.git'
13      }
14    }
15
16    stage('Build Docker Image') {
17      steps {
18        sh 'docker build -t $IMAGE_NAME .'
19      }
20    }
21
22    stage('Push Docker Image') {
23      steps {
24        sh '''
25          docker login -u iffah11 -p IffahMarch11
26          docker push $IMAGE_NAME
27          '''
28      }
29    }
30  }
31
32  post {
33    success {
34      echo 'Version 1 pipeline completed successfully'
35    }
36    failure {
37      echo 'Version 1 pipeline failed'
38    }
39  }
40 }
41 }
```

Figure 27: Jenkins pipeline job creation

The screenshot shows the Jenkins 'Stage View' for the 'IffahApp-V1' pipeline. The pipeline is in a successful state, indicated by a green checkmark. The 'Stage View' table shows the following stages and their durations:

Stage	Duration
Checkout	694ms
Build Docker Image	333ms
Push Docker Image	7s
Declarative: Post Actions	83ms

The 'Permalinks' section lists the following links:

- Last build (#9), 1 min 23 sec ago
- Last stable build (#9), 1 min 23 sec ago
- Last successful build (#9), 1 min 23 sec ago
- Last failed build (#8), 49 min ago
- Last unsuccessful build (#8), 49 min ago
- Last completed build (#9), 1 min 23 sec ago

Figure 28: Jenkins pipeline execution stages

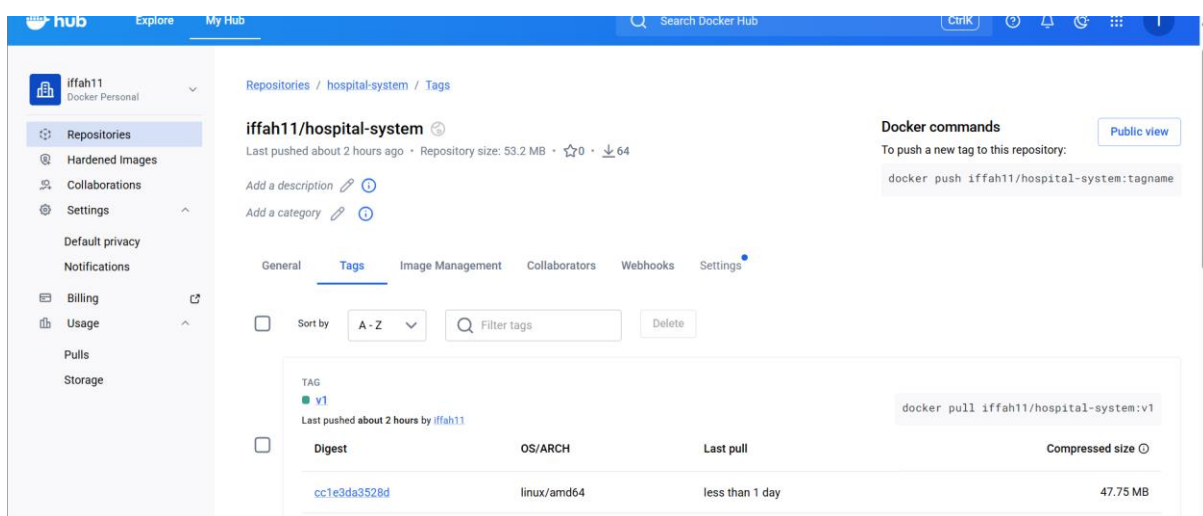
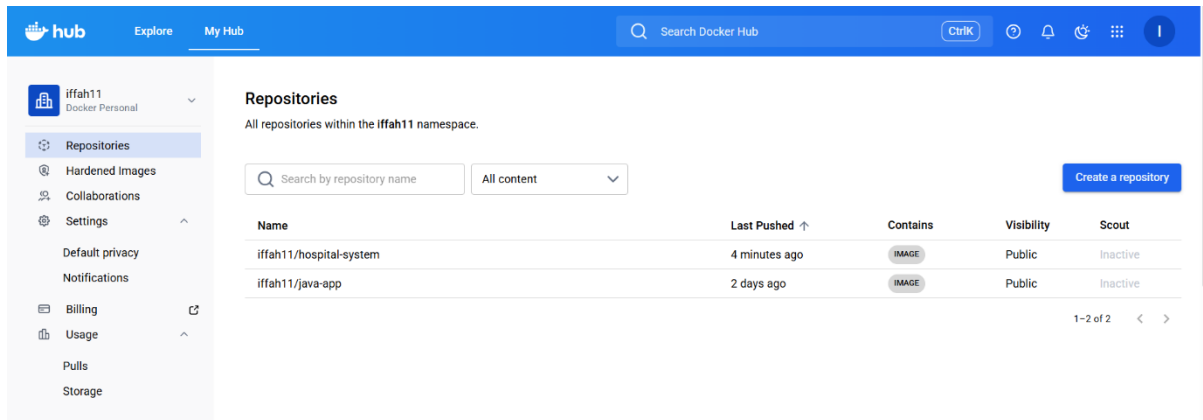


Figure 29: DockerHub showing v1 image

Phase 10 — Kubernetes Deployment

Kubernetes deployment was attempted manually using deployment and service configuration files.

```
$ kubectl apply -f k8s/v1-deployment.yaml
kubectl apply -f k8s/v1-service.yaml
deployment.apps/hospital-v1 configured
service/hospital-v1-service unchanged

Lenovo@LAPTOP-OFUS820G MINGW64 /d/Home/Iffah/Projects/hospital/hospital_system (v2-feature-view-patient)
$ kubectl get deployments
$ kubectl get pods
$ kubectl get services
NAME      READY   UP-TO-DATE   AVAILABLE   AGE
hospital-v1 1/1      1            1           3m54s
NAME      READY   STATUS    RESTARTS   AGE
hospital-v1-c88566cdb-jxtzt 1/1      Running    0           10s
NAME      TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
hospital-v1-service NodePort    10.101.110.235 <none>        5000:30081/TCP   3m54s
kubernetes ClusterIP  10.96.0.1     <none>        443/TCP         9m
```

Figure 30: kubectl apply commands and get resources

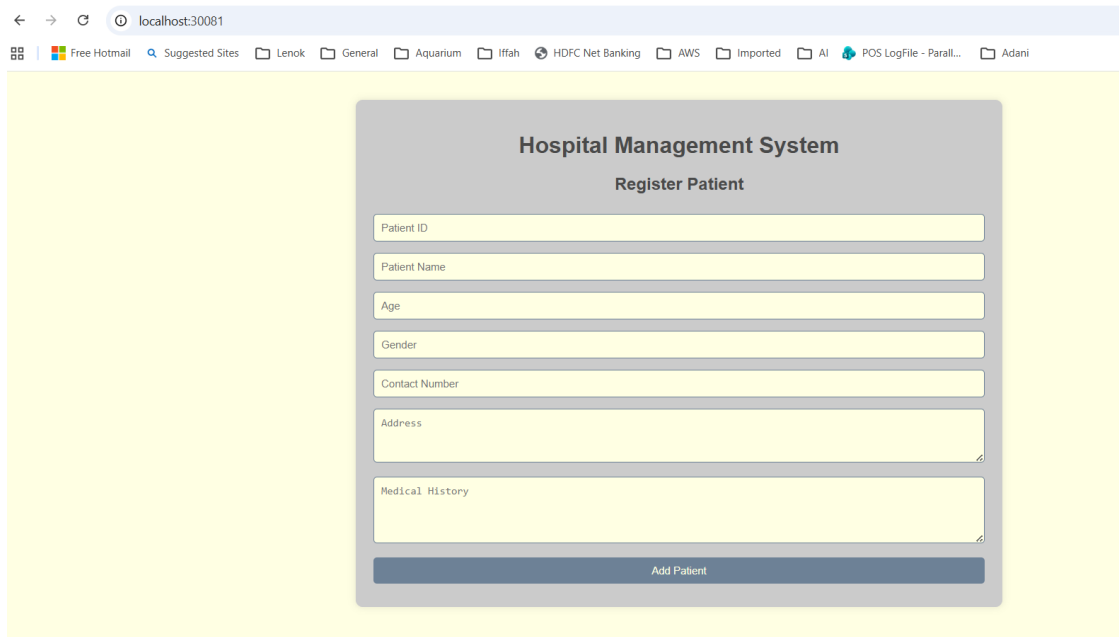


Figure 31: Deployed system on port 30081

VERSION 2 – VIEW PATIENTS

Version 2 enhances the system by introducing the ability to view stored patient records. A new page was added to display all patient data in a structured format.

Phase 1 — Branch Creation

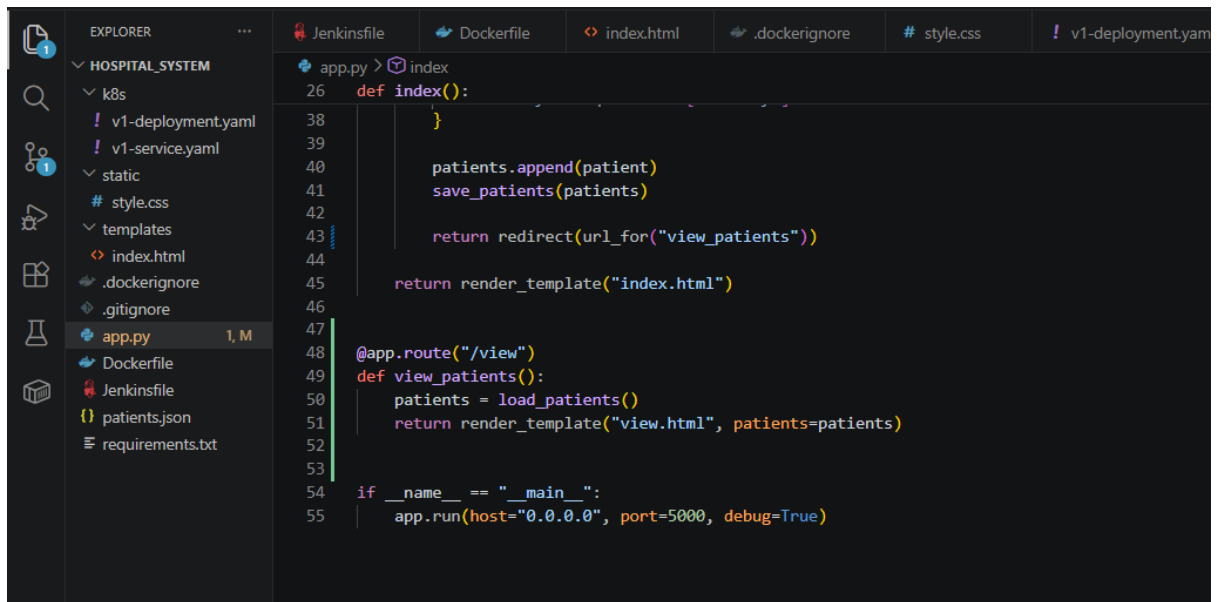
A new branch for Version 2 was created from the Version 1 branch to extend the existing functionality without affecting the previous version.

```
PS D:\Home\College\wint3\AGILE_CASE_STUDY\hospital_system> git checkout v1-feature-add-patient
>> git checkout -b v2-feature-view-patient
Switched to branch 'v1-feature-add-patient'
Your branch is up to date with 'origin/v1-feature-add-patient'.
Switched to a new branch 'v2-feature-view-patient'
PS D:\Home\College\wint3\AGILE_CASE_STUDY\hospital_system>
```

Figure 32: v2 branch created from v1

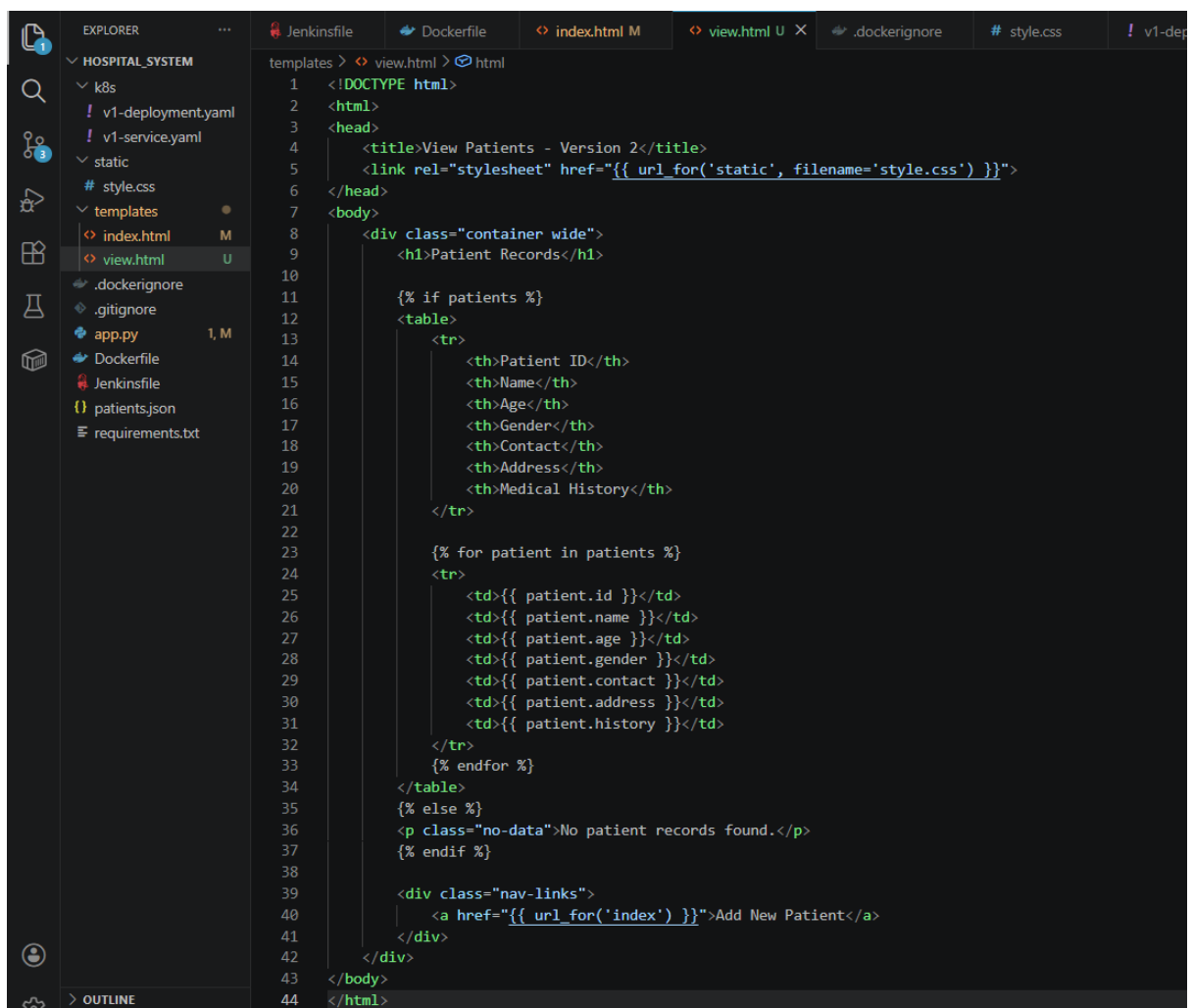
Phase 2 — Code Updates

The application was updated to include a new route for viewing patient records and a separate page to display the data. Navigation was added to move between pages easily.



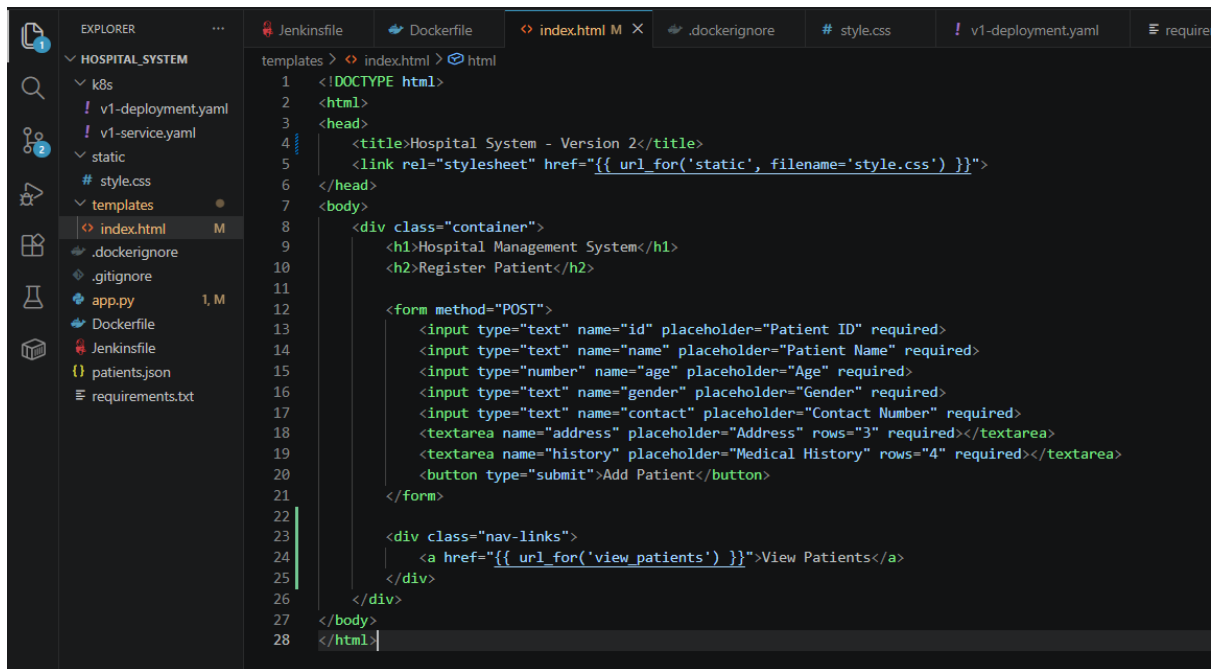
```
26 def index():
38     }
39
40     patients.append(patient)
41     save_patients(patients)
42
43     return redirect(url_for("view_patients"))
44
45     return render_template("index.html")
46
47
48 @app.route("/view")
49 def view_patients():
50     patients = load_patients()
51     return render_template("view.html", patients=patients)
52
53
54 if __name__ == "__main__":
55     app.run(host="0.0.0.0", port=5000, debug=True)
```

Figure 33: Updated app.py for v2



```
1 <!DOCTYPE html>
2 <html>
3 <head>
4     <title>View Patients - Version 2</title>
5     <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
6 </head>
7 <body>
8     <div class="container wide">
9         <h1>Patient Records</h1>
10
11         {% if patients %}
12         <table>
13             <tr>
14                 <th>Patient ID</th>
15                 <th>Name</th>
16                 <th>Age</th>
17                 <th>Gender</th>
18                 <th>Contact</th>
19                 <th>Address</th>
20                 <th>Medical History</th>
21             </tr>
22
23             {% for patient in patients %}
24             <tr>
25                 <td>{{ patient.id }}</td>
26                 <td>{{ patient.name }}</td>
27                 <td>{{ patient.age }}</td>
28                 <td>{{ patient.gender }}</td>
29                 <td>{{ patient.contact }}</td>
30                 <td>{{ patient.address }}</td>
31                 <td>{{ patient.history }}</td>
32             </tr>
33             {% endfor %}
34         </table>
35         {% else %}
36         <p class="no-data">No patient records found.</p>
37         {% endif %}
38
39         <div class="nav-links">
40             <a href="{{ url_for('index') }}">Add New Patient</a>
41         </div>
42     </div>
43 </body>
44 </html>
```

Figure 34: New view.html page



```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>Hospital System - Version 2</title>
5   <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
6 </head>
7 <body>
8   <div class="container">
9     <h1>Hospital Management System</h1>
10    <h2>Register Patient</h2>
11
12    <form method="POST">
13      <input type="text" name="id" placeholder="Patient ID" required>
14      <input type="text" name="name" placeholder="Patient Name" required>
15      <input type="number" name="age" placeholder="Age" required>
16      <input type="text" name="gender" placeholder="Gender" required>
17      <input type="text" name="contact" placeholder="Contact Number" required>
18      <textarea name="address" placeholder="Address" rows="3" required></textarea>
19      <textarea name="history" placeholder="Medical History" rows="4" required></textarea>
20      <button type="submit">Add Patient</button>
21    </form>
22
23    <div class="nav-links">
24      <a href="{{ url_for('view_patients') }}">View Patients</a>
25    </div>
26  </div>
27 </body>
28 </html>
```

Figure 35: Updated navigation

Phase 3 — Local Testing

The view functionality was tested to ensure that all stored patient records are displayed correctly and navigation between pages works as expected.

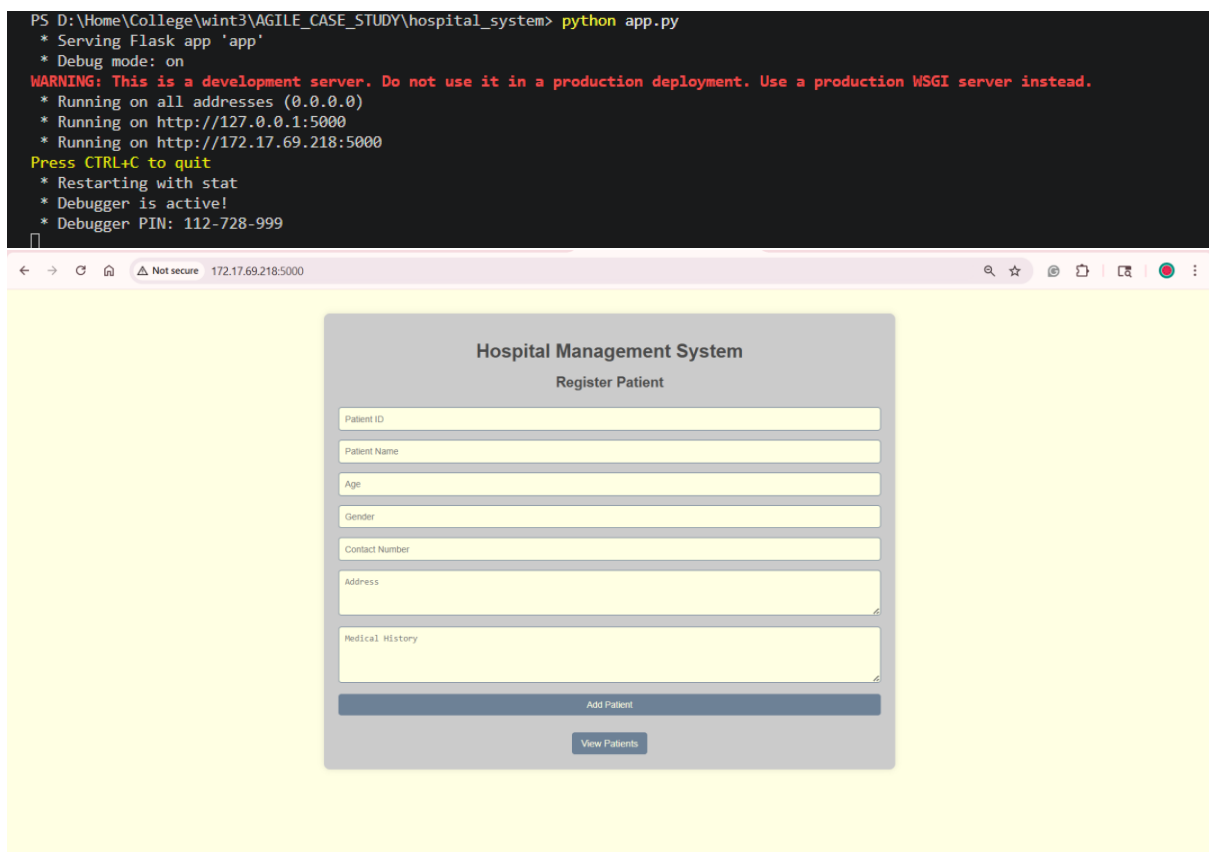


Figure 36: running the application on <http://127.0.0.1:5000>



Figure 37: the view page which displays all patient details

Phase 4 — Git Commit

All changes for Version 2 were committed and pushed to GitHub in a separate branch.

```
PS D:\Home\College\wint3\AGILE_CASE_STUDY\hospital_system> git add .
>> git commit -m "feat(v2): add view patient functionality"
>> git push -u origin v2-feature-view-patient
[v2-feature-view-patient 40d1a72] feat(v2): add view patient functionality
 4 files changed, 111 insertions(+), 2 deletions(-)
 create mode 100644 templates/view.html
Enumerating objects: 14, done.
Counting objects: 100% (14/14), done.
Delta compression using up to 8 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (8/8), 1.47 KiB | 188.00 KiB/s, done.
Total 8 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), completed with 4 local objects.
remote:
remote: Create a pull request for 'v2-feature-view-patient' on GitHub by visiting:
remote:   https://github.com/iffah-shafeeqe/hospital_system/pull/new/v2-feature-view-patient
remote:
To https://github.com/iffah-shafeeqe/hospital_system
 * [new branch]      v2-feature-view-patient -> v2-feature-view-patient
branch 'v2-feature-view-patient' set up to track 'origin/v2-feature-view-patient'.
PS D:\Home\College\wint3\AGILE_CASE_STUDY\hospital_system>
```

Figure 38: Git commit for v2

Phase 5 — Docker Build

A new Docker image for Version 2 was built and tested locally on a different port.

```
$ docker build -t v2-feature-view-patients .
[+] Building 11.5s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> transferring dockerfile: 210B
=> [internal] load metadata for docker.io/library/python:3.13-slim
=> [internal] load .dockerignore
=> transferring context: 2B
=> [1/5] FROM docker.io/library/python:3.13-slim@sha256:d168b8d9eb761f4d3fe385ebd04aeb7e7f2de0297cec5fb2f8f6403244621664
=> resolve docker.io/library/python:3.13-slim@sha256:d168b8d9eb761f4d3fe385ebd04aeb7e7f2de0297cec5fb2f8f6403244621664
=> sha256:baa1df13ceddd69aaa907923f91541b3d2d1a10ea092f411b2b4d75cad027d 250B / 250B
=> sha256:7e1c75a1cbdd706165f10aa82e8a53bfc4819367c8a5b34265f74c481274412 11.82MB / 11.82MB
=> sha256:c841cd3d90201032366c4f50f468dce928a1f9b0f43552c3890091c01a9ce8dc 4.26MB / 4.26MB
=> extracting sha256:c841cd3d90201032366c4f50f468dce928a1f9b0f43552c3890091c01a9ce8dc
=> extracting sha256:7e1c75a1cbdd706165f10aa82e8a53bfc4819367c8a5b34265f74c481274412
=> extracting sha256:baa1df13ceddd69aaa907923f91541b3d2d1a10ea092f411b2b4d75cad027d
=> [internal] load build context
=> transferring context: 46.79kB
=> [2/5] WORKDIR /app
=> [3/5] COPY requirements.txt .
=> [4/5] RUN pip install --no-cache-dir -r requirements.txt
=> [5/5] COPY . .
=> exporting to image
=> exporting layers
=> exporting manifest sha256:a4657b744acee64479751506b4e21a8b1bdd9c4fc55be67alc64c870c6a2d4c
=> exporting config sha256:cb71ab26258408010c2edc9e2647b26eda1358c19a26b1bc6d216a8843b4a9e6
=> exporting attestation manifest sha256:5da9ab44bd5490327f9eacbc122e090d9339a6570f1783378b98311817ae1ddc3
=> exporting manifest list sha256:d4493958f84bed3d4fa50f574f43b6d9e9d9f83c0b1887c3e718db7996f6d0bb
=> naming to docker.io/library/v2-feature-view-patients:latest
=> unpacking to docker.io/library/v2-feature-view-patients:latest
What's next:
  View a summary of image vulnerabilities and recommendations → docker scout quickview
$ docker run -p 5002:5000 v2-feature-view-patients
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://172.17.0.1:5000
* Running on http://172.17.0.2:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 643-042-228
172.17.0.1 - - [12/Apr/2026 07:52:21] "GET / HTTP/1.1" 200 -
172.17.0.1 - - [12/Apr/2026 07:52:21] "GET /static/style.css HTTP/1.1" 200 -
172.17.0.1 - - [12/Apr/2026 07:52:21] "GET /favicon.ico HTTP/1.1" 404 -
```

Figure 39: Docker build for v2

← → ↻ localhost:5002

Free Hotmail Suggested Sites Lenok General Aquarium Iffah HDFC Net Banking AWS Imported AI POS LogFile - Parall... Adani

Hospital Management System

Register Patient

Patient ID

Patient Name

Age

Gender

Contact Number

Address

Medical History

Add Patient

View Patients

← → ↻ localhost:5002/view

Free Hotmail Suggested Sites Lenok General Aquarium Iffah HDFC Net Banking AWS Imported AI POS LogFile - Parall... Adani All Bookmarks

Patient Records

Patient ID	Name	Age	Gender	Contact	Address	Medical History
P1001	Iffah Shafesque	22	Female	9993728292	Kozhikode, Kerala	Diabetes, seasonal allergies

Add New Patient

Figure 40: Application running on localhost:5002

Phase 6 — Jenkins Pipeline

The Jenkins pipeline was executed again to automate the build and push process for Version 2.

 Jenkins / IffahApp-V2 / Configure

Configure

General

Triggers

Pipeline

Advanced

Pipeline

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script

Script ?

```
1 pipeline {
2   agent any
3
4   environment {
5     IMAGE_NAME = "iffah11/hospital-system:v2"
6   }
7
8   stages {
9     stage('Checkout') {
10      steps {
11        git branch: 'v2-feature-view-patient',
12           url: 'https://github.com/iffah-shafeeqe/hospital_system.git'
13      }
14    }
15
16    stage('Build Docker Image') {
17      steps {
18        sh 'docker build -t $IMAGE_NAME .'
19      }
20    }
21
22    stage('Push Docker Image') {
23      steps {
24        sh '''
25          docker login -u iffah11 -p IffahMarch11
26          docker push $IMAGE_NAME
27        '''
28      }
29    }
30  }
31
32  post {
33    success {
34      echo 'Version 1 pipeline completed successfully'
35    }
36    failure {
37      echo 'Version 1 pipeline failed'
38    }
39  }
40 }
41
42 }
```

 Jenkins / IffahApp-V2

Status

Changes

Build Now

Configure

Delete Pipeline

Move

Full Stage View

Job Config History

Rename

Pipeline Syntax

IffahApp-V2

Stage View

Average stage times:
(Average full run time: ~15s)

Checkout	Build Docker Image	Push Docker Image	Declarative: Post Actions
679ms	1s	13s	81ms

#2 Apr 12 15:17 1 commit

Checkout	Build Docker Image	Push Docker Image	Declarative: Post Actions
679ms	1s	13s	81ms

Permalinks

- Last build (#2), 43 min ago
- Last stable build (#2), 43 min ago
- Last successful build (#2), 43 min ago
- Last completed build (#2), 43 min ago

Builds >

Filter

Today

#2 9:47 AM

Figure 41: Jenkins pipeline for v2

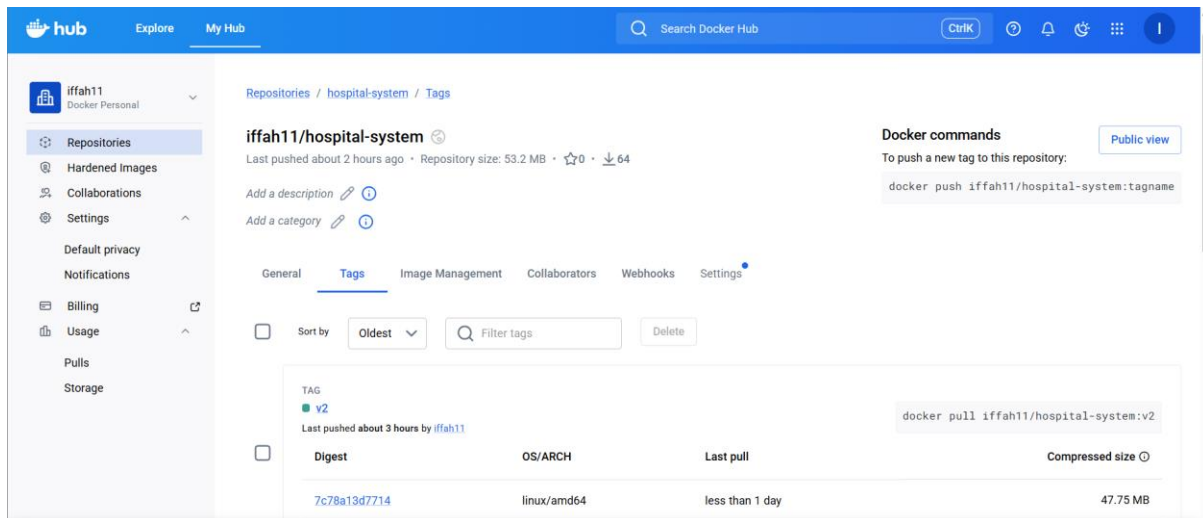


Figure 42: DockerHub showing v2 image

Phase 7 — Kubernetes

Deployment configuration files were prepared for Version 2 and Kubernetes deployment was done.

```

Code  Blame  20 lines (20 loc) · 388 Bytes

1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: hospital-v2
5  spec:
6    replicas: 1
7    selector:
8      matchLabels:
9        app: hospital-v2
10   template:
11     metadata:
12       labels:
13         app: hospital-v2
14     spec:
15       containers:
16       - name: hospital-v2
17         image: iffah11/hospital-system:v2
18         imagePullPolicy: Always
19         ports:
20         - containerPort: 5000

```

Figure 43: v2 deployment.yaml

```

Code Blame 13 lines (13 loc) · 213 Bytes

1  apiVersion: v1
2  kind: Service
3  metadata:
4    name: hospital-v2-service
5  spec:
6    type: NodePort
7    selector:
8      app: hospital-v2
9    ports:
10     - protocol: TCP
11       port: 5000
12       targetPort: 5000
13       nodePort: 30082

```

Figure 44: v2 service.yaml

```

Lenovo@LAPTOP-0FUS820G MINGW64 /d/Home/Iffah/Projects/hospital/hospital_system (v2-feature-view-patient)
$ kubectl apply -f k8s/v2-deployment.yaml
kubectl apply -f k8s/v2-service.yaml
deployment.apps/hospital-v2 configured
service/hospital-v2-service configured

Lenovo@LAPTOP-0FUS820G MINGW64 /d/Home/Iffah/Projects/hospital/hospital_system (v2-feature-view-patient)
$ kubectl get deployments
kubectl get pods
kubectl get services

```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
hospital-v1	1/1	1	1	16m
hospital-v2	1/1	1	1	4m6s

NAME	READY	STATUS	RESTARTS	AGE
hospital-v1-c88566cdb-jxtzt	1/1	Running	0	12m
hospital-v2-676cbbbbc-m5nx2	1/1	Running	0	119s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
hospital-v1-service	NodePort	10.101.110.235	<none>	5000:30081/TCP	16m
hospital-v2-service	NodePort	10.100.58.102	<none>	5000:30082/TCP	4m7s
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	21m

Figure 45: kubectl apply commands and get resources

The screenshot shows a web browser window with the address bar set to localhost:30082. The browser's address bar and tabs are visible at the top. The main content area displays a web application titled "Hospital Management System" with a subtitle "Register Patient". The application form includes several input fields: "Patient ID", "Patient Name", "Age", "Gender", "Contact Number", "Address", and "Medical History". At the bottom of the form, there are two buttons: "Add Patient" and "View Patients". The background of the page is a solid light yellow color.

Figure 46: Deployed system on port 30082

VERSION 3 – DELETE PATIENT

Version 3 completes the system by adding the ability to delete patient records directly from the view page.

Phase 1 — Branch Creation

A Version 3 branch was created from Version 2 to extend the system with deletion functionality.

```
PS D:\Home\College\wint3\AGILE CASE STUDY\hospital system> git checkout v2-feature-view-patient
>> git checkout -b v3-feature-delete-patient
Switched to branch 'v2-feature-view-patient'
Your branch is up to date with 'origin/v2-feature-view-patient'.
fatal: a branch named 'v3-feature-delete-patient' already exists
```

Figure 47: v3 branch created from v2

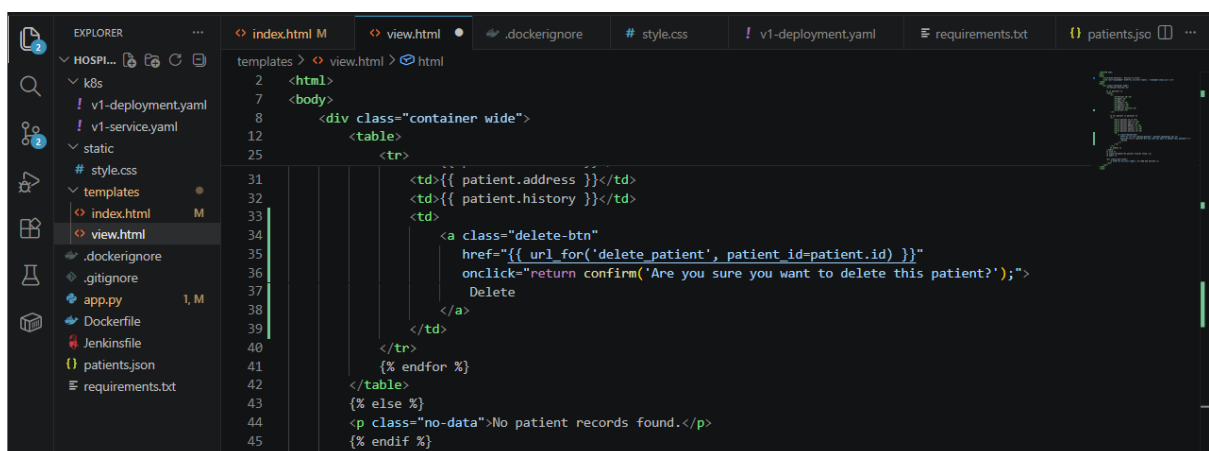
Phase 2 — Code Updates

The application was updated to include a delete route and corresponding user interface elements. A delete button was added for each patient record.



```
app.py > ...
26 def index():
41     save_patients(patients)
42
43     return redirect(url_for("view_patients"))
44
45     return render_template("index.html")
46
47
48 @app.route("/view")
49 def view_patients():
50     patients = load_patients()
51     return render_template("view.html", patients=patients)
52
53
54 @app.route("/delete/<patient_id>")
55 def delete_patient(patient_id):
56     patients = load_patients()
57     updated_patients = [patient for patient in patients if patient["id"] != patient_id]
58     save_patients(updated_patients)
59     return redirect(url_for("view_patients"))
60
61
62 if __name__ == "__main__":
63     app.run(host="0.0.0.0", port=5000, debug=True)
```

Figure 48: Updated app.py with delete functionality



```
templates > view.html > html
2 <html>
7 <body>
8 <div class="container wide">
12 <table>
25 <tr>
31 <td>{{ patient.address }}</td>
32 <td>{{ patient.history }}</td>
33 <td>
34 <a class="delete-btn"
35 href="{{ url_for('delete_patient', patient_id=patient.id) }}"
36 onclick="return confirm('Are you sure you want to delete this patient?');">
37 Delete
38 </a>
39 </td>
40 </tr>
41 <tr>
42 <td colspan="3">{% endfor %}</td>
43 </table>
44 <div class="no-data">No patient records found.</div>
45 </div>
46 </body>
47 </html>
```

Figure 49: view.html updated with delete button

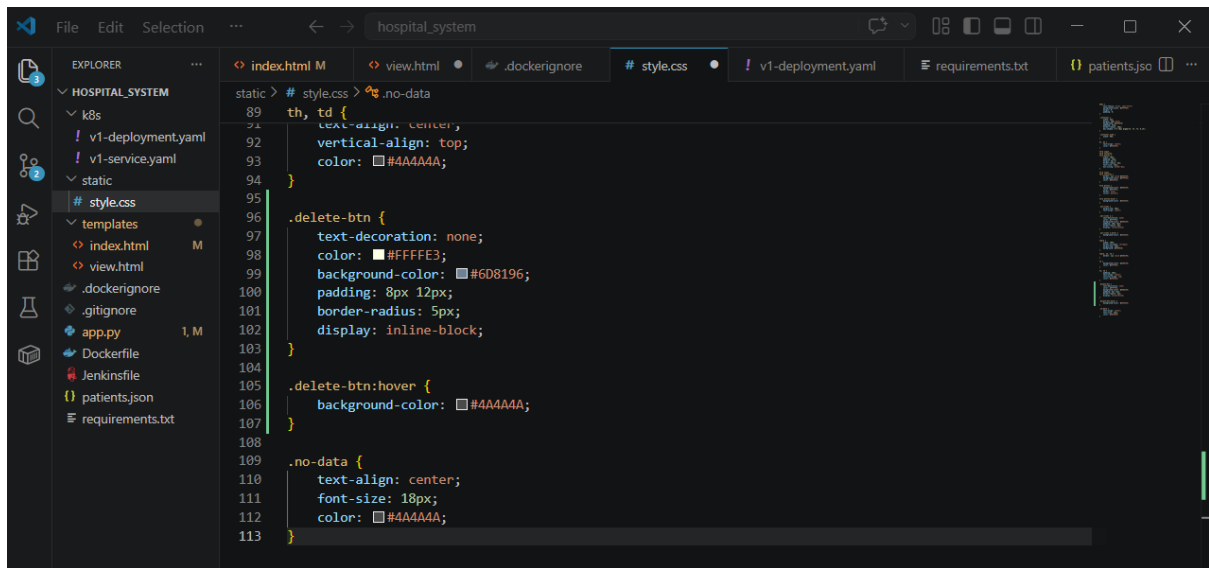


Figure 50: Updated UI with delete option

Phase 3 — Local Testing

The delete functionality was tested to ensure that records are removed correctly and the updated list is displayed.

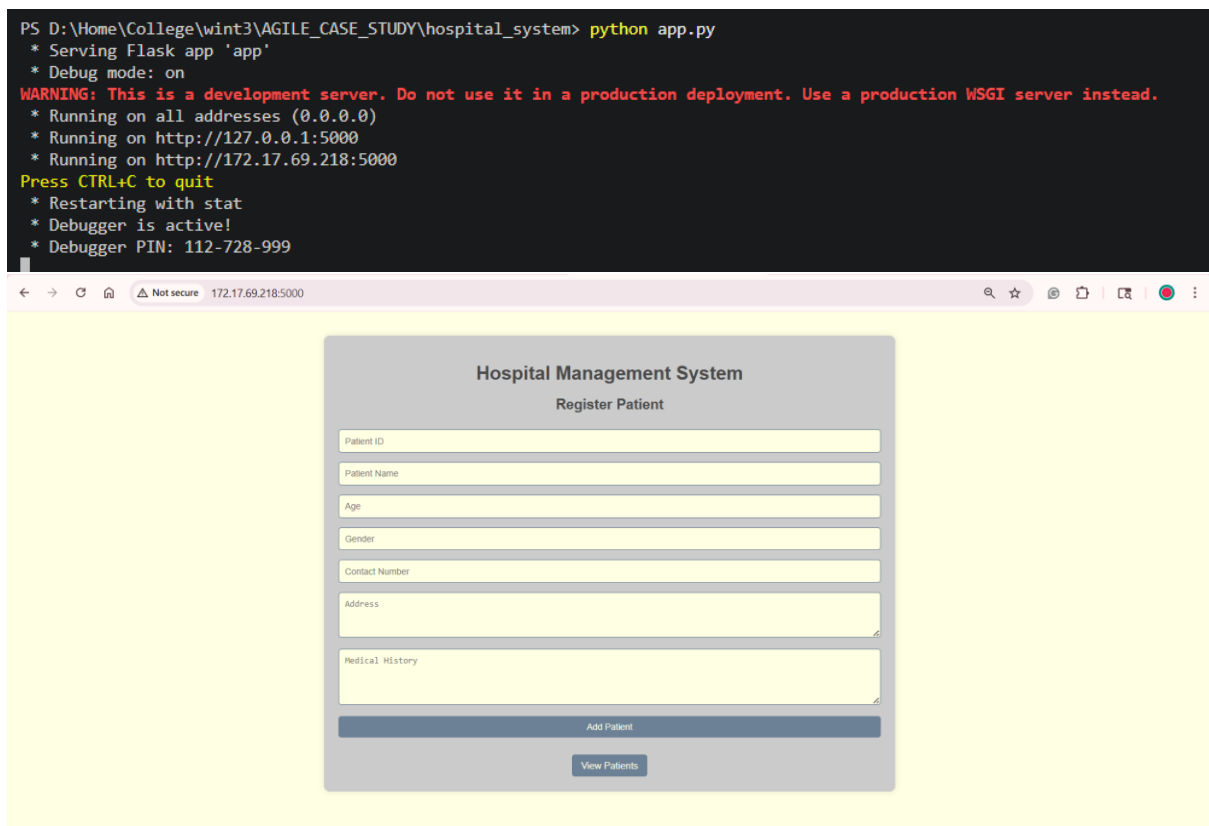


Figure 51: App ran on <http://127.0.0.1:5000>

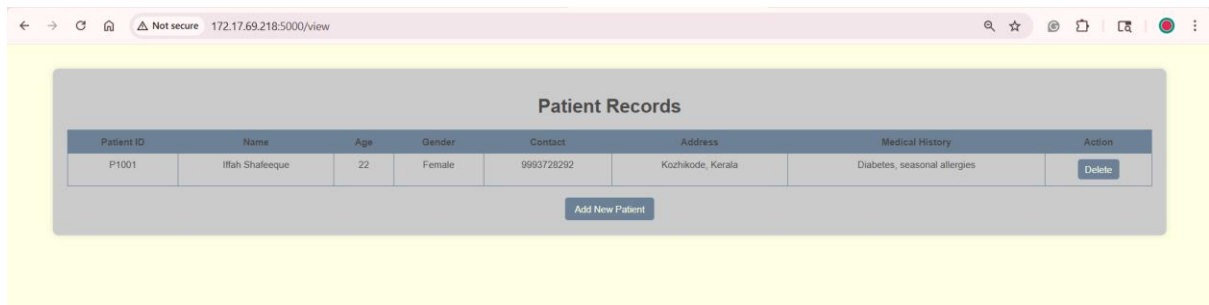


Figure 52: Patient list before deletion

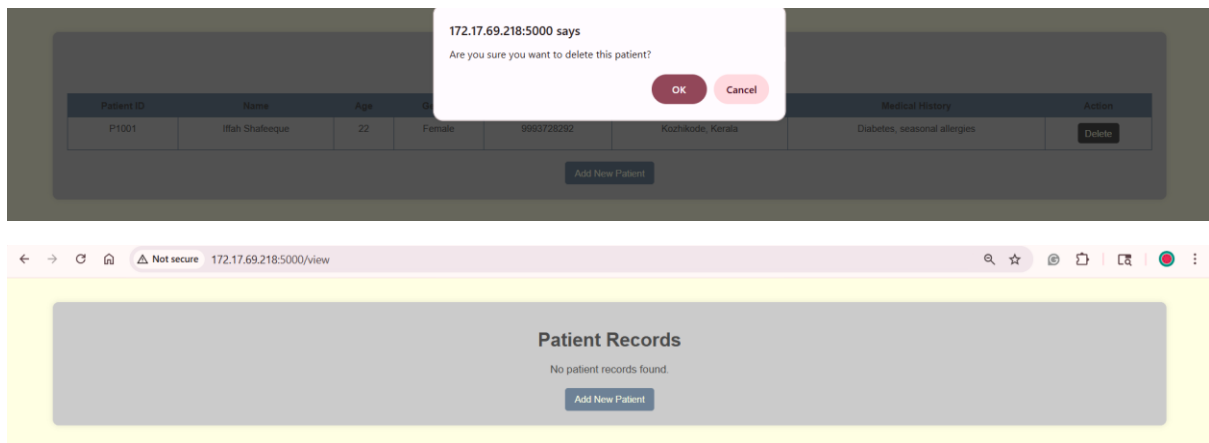


Figure 53: Patient list after deletion

Phase 4 — Git Commit

All Version 3 changes were committed and pushed to GitHub.

```
PS D:\Home\College\wint3\AGILE_CASE_STUDY\hospital_system> git add .
>> git commit -m "feat(v3): add delete patient functionality"
>> git push -u origin v3-feature-delete-patient
[v2-feature-view-patient 69ce3ad] feat(v3): add delete patient functionality
3 files changed, 22 insertions(+), 1 deletion(-)
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'v3-feature-delete-patient' on GitHub by visiting:
remote:   https://github.com/iffah-shafeeqe/hospital_system/pull/new/v3-feature-delete-patient
remote:
To https://github.com/iffah-shafeeqe/hospital_system
 * [new branch]      v3-feature-delete-patient -> v3-feature-delete-patient
branch 'v3-feature-delete-patient' set up to track 'origin/v3-feature-delete-patient'.
PS D:\Home\College\wint3\AGILE_CASE_STUDY\hospital_system>
```

Figure 54: Git commit for v3

Phase 5 — Docker Build

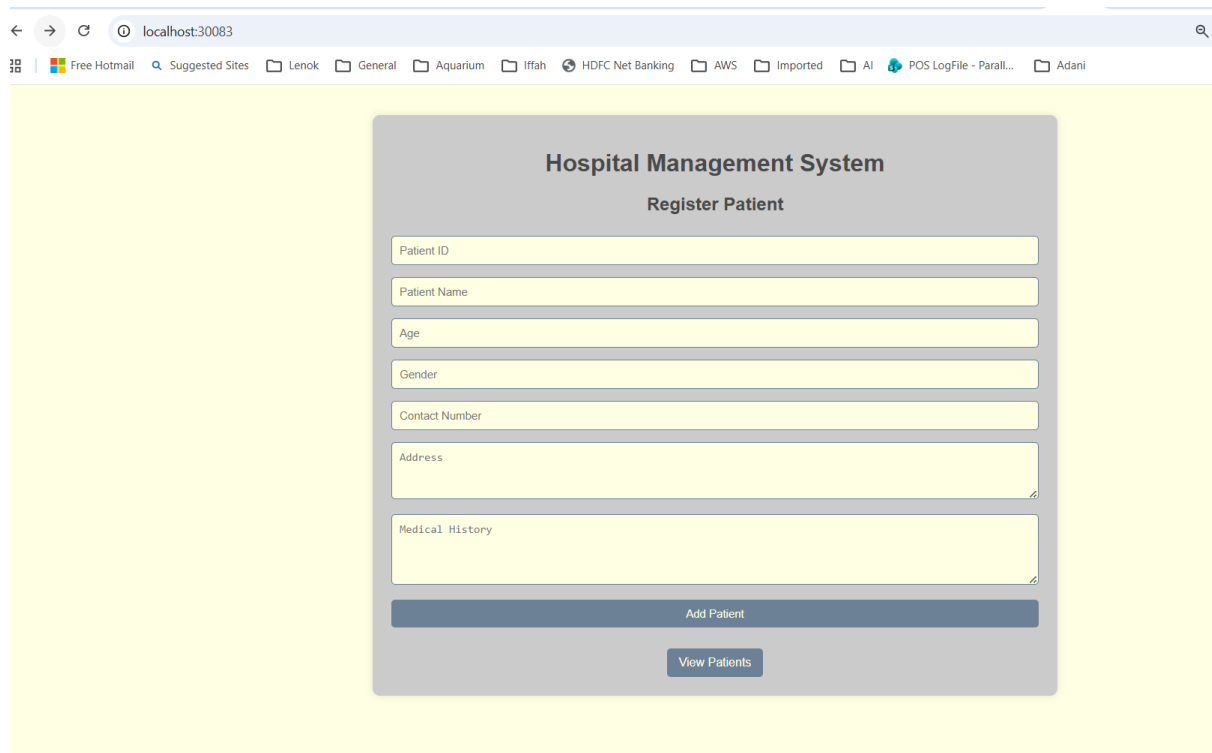
A Docker image for Version 3 was built and tested locally.

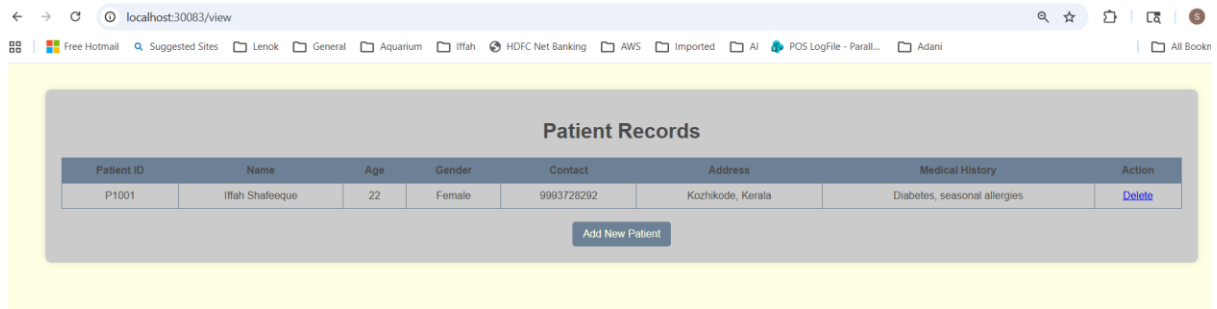
```
$ docker build -t v3-feature-delete-patient .
[+] Building 2.4s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 210B
=> [internal] load metadata for docker.io/library/python:3.11-slim
=> [internal] load .dockerignore
=> => transferring context: 107B
=> [1/5] FROM docker.io/library/python:3.11-slim@sha256:233de06753d30d120b1a3ce359d8d3be8bda78524cd8f520c99883bfe33964cf
=> => resolve docker.io/library/python:3.11-slim@sha256:233de06753d30d120b1a3ce359d8d3be8bda78524cd8f520c99883bfe33964cf
=> [internal] load build context
=> => transferring context: 2.01kB
=> CACHED [2/5] WORKDIR /app
=> CACHED [3/5] COPY requirements.txt
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt
=> [5/5] COPY .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:16438ef2b90c3e75c8acbe15aac8340c0a38ddade5b7fd2c63ff49089d5572a
=> => exporting config sha256:82bba0ebc73b2665d04f088470ef6260a09d68e60f1f46b4a2bb43f85e731c8
=> => exporting attestation manifest sha256:cfc8705a80102483a469112e83cccaad13f8c94ee277cd0ef1d257b4759884b1
=> => exporting manifest list sha256:62466cd3008281a724290ba65807bba8cedef216fdc558707415fd074b667f6
=> => naming to docker.io/library/v3-feature-delete-patient:latest
=> => unpacking to docker.io/library/v3-feature-delete-patient:latest

What's next:
  View a summary of image vulnerabilities and recommendations + docker scout quickview

$ docker run -p 5000:5000 v3-feature-delete-patient
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.17.0.2:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 753-937-290
172.17.0.1 - - [12/Apr/2026 09:13:39] "GET /view HTTP/1.1" 200 -
172.17.0.1 - - [12/Apr/2026 09:13:39] "GET /static/style.css HTTP/1.1" 304 -
```

Figure 55: Docker build for v3





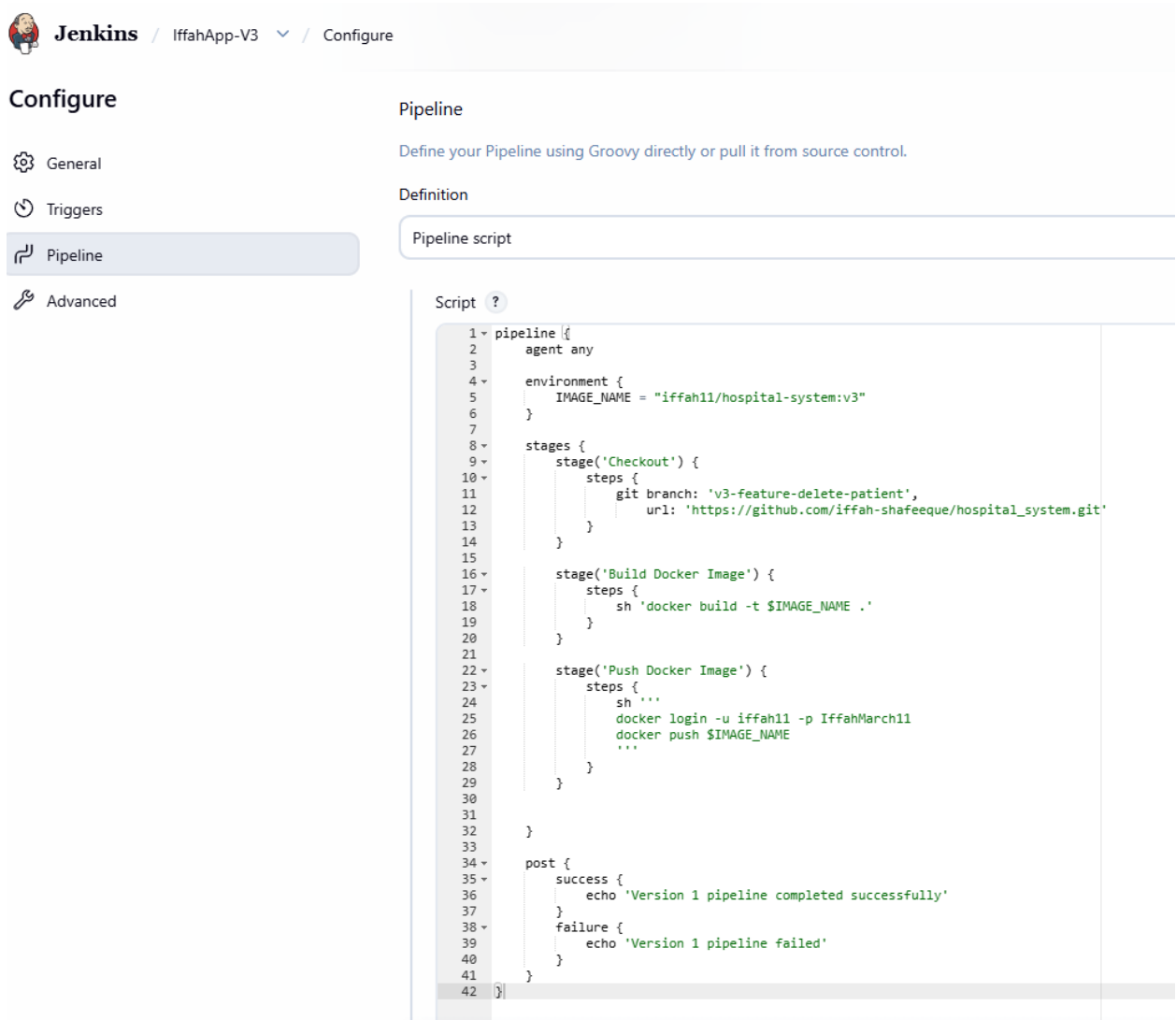
Patient ID	Name	Age	Gender	Contact	Address	Medical History	Action
P1001	Ifrah Shafeeqe	22	Female	9993728292	Kozhikode, Kerala	Diabetes, seasonal allergies	Delete

[Add New Patient](#)

Figure 56: Application running on localhost:5003

Phase 6 — Jenkins Pipeline

The pipeline was executed for Version 3 to automate image creation and push.



Jenkins / IffahApp-V3 / Configure

Configure

- General
- Triggers
- Pipeline**
- Advanced

Pipeline

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script

```

1 pipeline {
2   agent any
3
4   environment {
5     IMAGE_NAME = "iffah11/hospital-system:v3"
6   }
7
8   stages {
9     stage('Checkout') {
10      steps {
11        git branch: 'v3-feature-delete-patient',
12            url: 'https://github.com/iffah-shafeeqe/hospital_system.git'
13      }
14    }
15
16    stage('Build Docker Image') {
17      steps {
18        sh 'docker build -t $IMAGE_NAME .'
19      }
20    }
21
22    stage('Push Docker Image') {
23      steps {
24        sh '''
25          docker login -u iffah11 -p IffahMarch11
26          docker push $IMAGE_NAME
27        '''
28      }
29    }
30
31  }
32
33  post {
34    success {
35      echo 'Version 1 pipeline completed successfully'
36    }
37    failure {
38      echo 'Version 1 pipeline failed'
39    }
40  }
41 }
42

```


Jenkins / IffahApp-V3

Status

Changes

Build Now

Configure

Delete Pipeline

Move

Full Stage View

Job Config History

Rename

Pipeline Syntax

IffahApp-V3

Stage View

	Checkout	Build Docker Image	Push Docker Image	Declarative: Post Actions
Average stage times: (Average full run time: ~15s)	881ms	1s	12s	78ms
#1 Apr 12 15:18 No Changes	881ms	1s	12s	78ms

Permalinks

- Last build (#1), 43 min ago
- Last stable build (#1), 43 min ago
- Last successful build (#1), 43 min ago
- Last completed build (#1), 43 min ago

Builds >

Filter

Today

#1 9:48 AM

Figure 57: Jenkins pipeline for v3

hub

Explore My Hub

Search Docker Hub

CtrlK

iffah11

Docker Personal

Repositories

Hardened Images

Collaborations

Settings

Default privacy

Notifications

Billing

Usage

Pulls

Storage

Repositories / hospital-system / Tags

iffah11/hospital-system

Last pushed about 3 hours ago • Repository size: 53.2 MB • ☆0 • 64

Add a description

Add a category

General

Tags

Image Management

Collaborators

Webhooks

Settings

Sort by

Z - A

Filter tags

Delete

TAG

v3

Last pushed about 3 hours by iffah11

Digest	OS/ARCH	Last pull	Compressed size
bec735b905a4	linux/amd64	less than 1 day	47.75 MB

Docker commands

Public view

To push a new tag to this repository:

docker push iffah11/hospital-system:tagname

docker pull iffah11/hospital-system:v3

Figure 58: DockerHub showing v3 image

Phase 7 — Kubernetes

Deployment files for Version 3 were prepared and Kubernetes deployed.

```
Code Blame 20 lines (20 loc) · 388 Bytes

1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: hospital-v3
5  spec:
6    replicas: 1
7    selector:
8      matchLabels:
9        app: hospital-v3
10   template:
11     metadata:
12       labels:
13         app: hospital-v3
14     spec:
15       containers:
16       - name: hospital-v3
17         image: iffah11/hospital-system:v3
18         imagePullPolicy: Always
19         ports:
20         - containerPort: 5000
```

Figure 58: v3 deployment.yaml

```
Code Blame 13 lines (13 loc) · 213 Bytes

1  apiVersion: v1
2  kind: Service
3  metadata:
4    name: hospital-v3-service
5  spec:
6    type: NodePort
7    selector:
8      app: hospital-v3
9    ports:
10     - protocol: TCP
11       port: 5000
12       targetPort: 5000
13     nodePort: 30083
```

Figure 59: v3 service.yaml

```
$
kubectl apply -f k8s/v3-deployment.yaml
kubectl apply -f k8s/v3-service.yaml
deployment.apps/hospital-v3 created
service/hospital-v3-service created

lenovo@LAPTOP-OFUS820G MINGW64 /d/Home/Ifpah/Projects/hospital/hospital_system (v2-feature-view-patient)
$ kubectl get deployments
kubectl get pods
kubectl get services
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
hospital-v1   1/1     1            1           19m
hospital-v2   1/1     1            1           6m33s
hospital-v3   1/1     1            1           17s
NAME          READY   STATUS    RESTARTS   AGE
hospital-v1-c88566cdb-jxtzt  1/1     Running   0           15m
hospital-v2-676cbbbbc-m5nx2  1/1     Running   0           4m26s
hospital-v3-59f777f98d-whmpv  1/1     Running   0           17s
NAME          TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
hospital-v1-service  NodePort    10.101.110.235 <none>        5000:30081/TCP   19m
hospital-v2-service  NodePort    10.100.58.102 <none>        5000:30082/TCP   6m34s
hospital-v3-service  NodePort    10.107.61.180 <none>        5000:30083/TCP   18s
kubernetes      ClusterIP   10.96.0.1    <none>        443/TCP          24m
```

Figure 60: kubectl apply commands and get resources

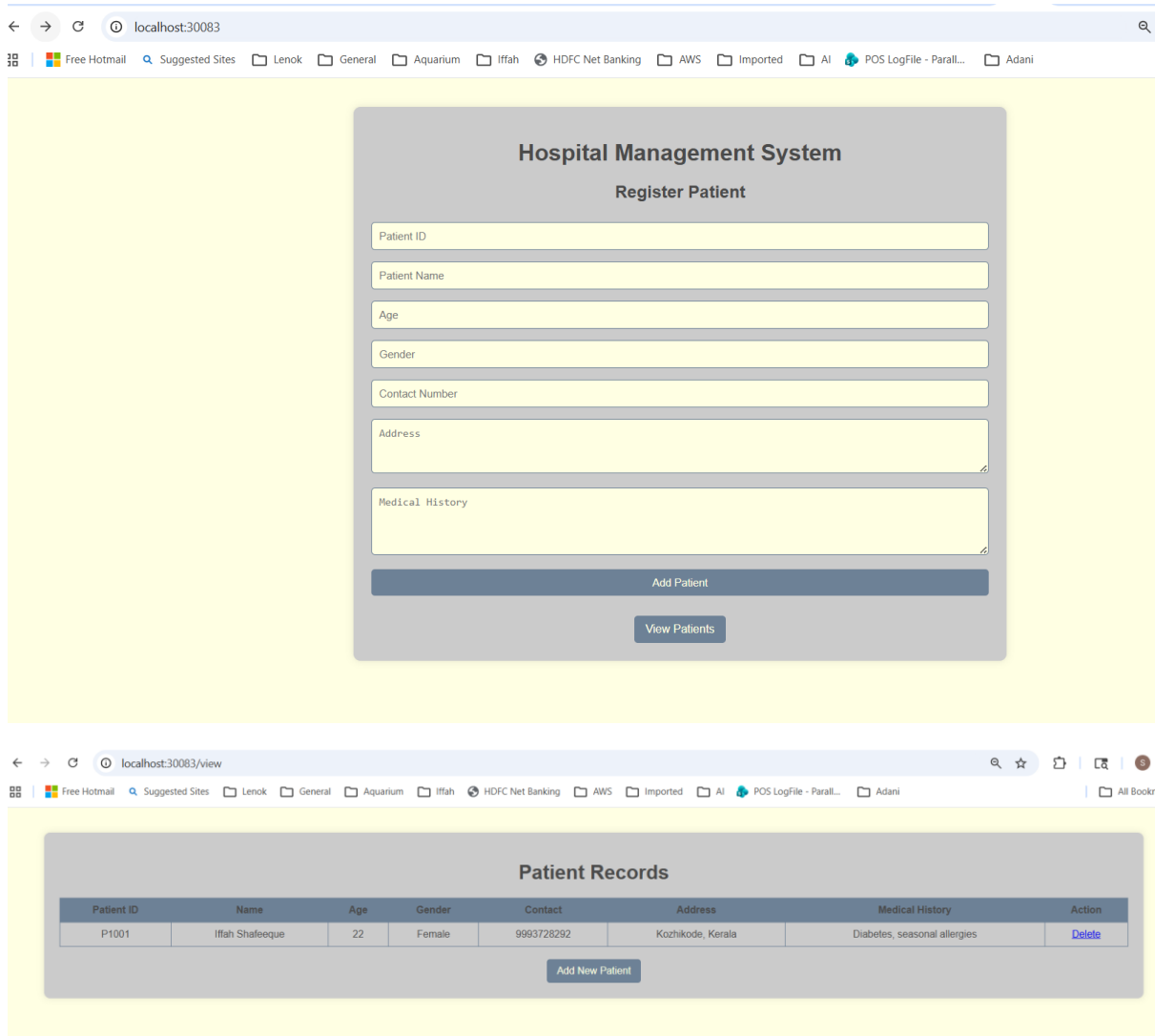


Figure 61: run on port 30083

CONCLUSION

This case study successfully demonstrates the development of a Hospital Management System through an incremental Agile approach combined with DevOps practices. Each version progressively enhanced the system by introducing new functionality while maintaining clear version control through branching. The integration of Docker and Jenkins enabled consistent containerization and automated build processes, while DockerHub was used for managing versioned images. Kubernetes configurations were prepared to support deployment across different versions. The project effectively showcases a complete development-to-deployment workflow, highlighting both technical implementation and process understanding. Overall, it provides a practical demonstration of how Agile and DevOps can be applied together in real-world software development.